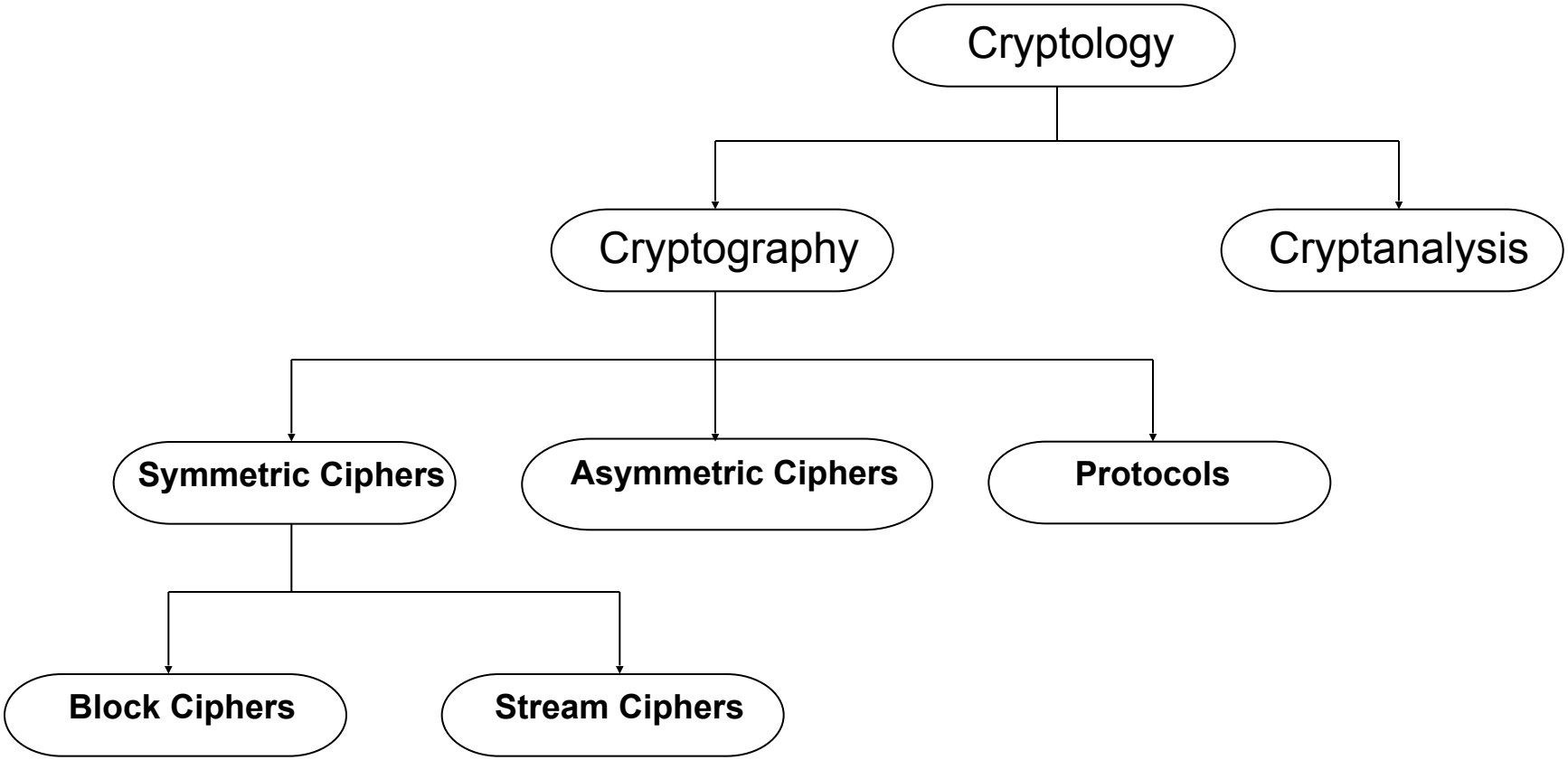
Hardware Security
硬體安全

Brief Introduction to Cryptography

Wei-Chih Hong

Classification of the Field of Cryptology

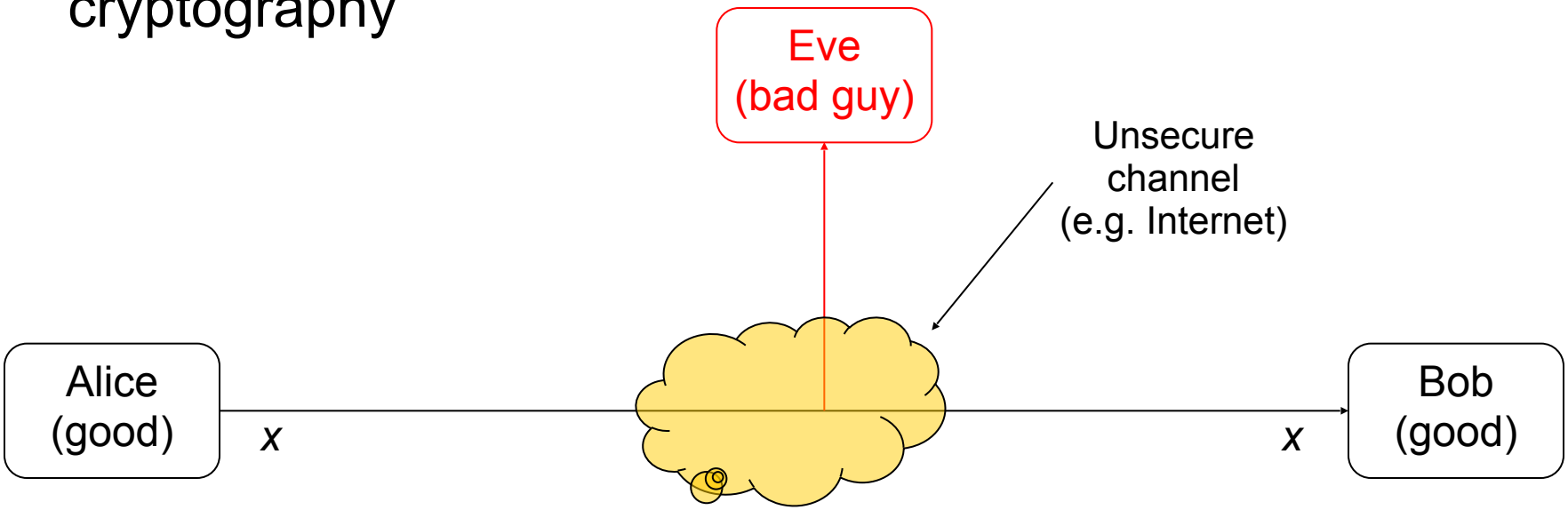


Some Basic Facts

- **Ancient Crypto:** Early signs of encryption in Egypt in ca. 2000 B.C. Letter-based encryption schemes (e.g., Caesar cipher) popular ever since
- **Symmetric ciphers:** All encryption schemes from ancient times until 1976 were symmetric ones
- **Asymmetric ciphers:** In 1976 public-key (or asymmetric) cryptography was openly proposed by Diffie, Hellman and Merkle
- **Hybrid Schemes:** The majority of today‘s protocols are hybrid schemes, i.e., the use both
  - symmetric ciphers (e.g., for encryption and message authentication)
  - asymmetric ciphers (e.g., for key exchange and digital signature)

Symmetric Cryptography

- Alternative names: **private-key**, **single-key** or **secret-key** cryptography

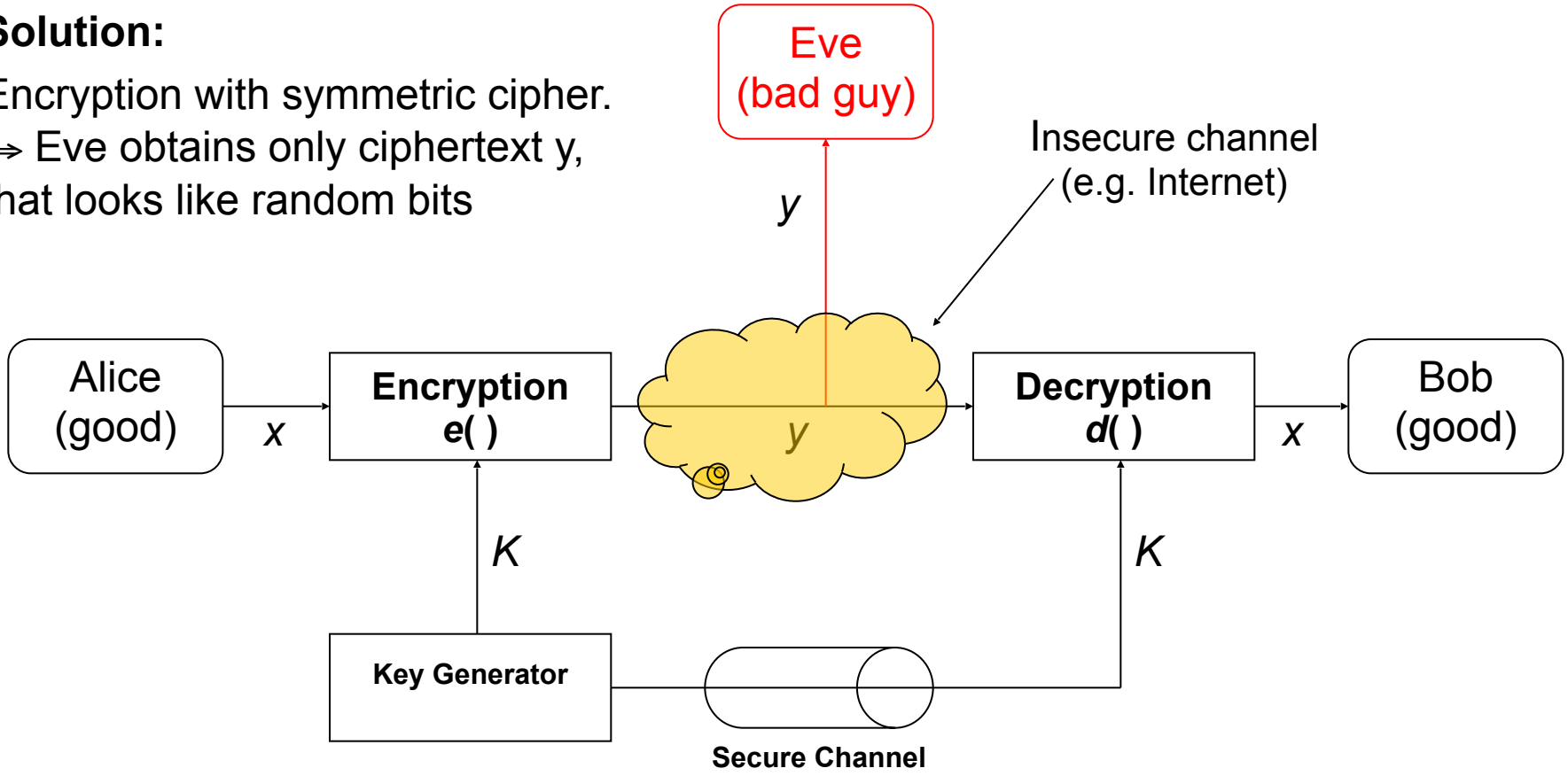


- **Problem Statement:**
  1) Alice and Bob would like to communicate via an insecure channel (e.g., WLAN or Internet)
  2) A malicious third party Eve (the bad guy) has channel access but should not be able to understand the communication

Symmetric Cryptography

Solution:

Encryption with symmetric cipher.
⇒ Eve obtains only ciphertext y,
that looks like random bits



- *x* is the **plaintext**
- *y* is the **ciphertext**
- *K* is the **key**
- Set of all keys $\{K_1, K_2, ..., K_n\}$ is the **key space**

Symmetric Cryptography

- Encryption equation $y = e_K(x)$
- Decryption equation $x = d_K(y)$

- Encryption and decryption are inverse operations if the same key *K* is used on both sides:

$$d_K(y) = d_K(e_K(x)) = x$$

- Important: The key must be transmitted via a **secure channel** between Alice and Bob
- The secure channel can be realized, e.g., by manually installing the key for the Wi-Fi Protected Access (WPA) protocol or a human courier
- However, the system is secure only if an attacker does not learn the key *K*

⇒ **The problem of secure communication is reduced to secure transmission and storage of the key *K***

Symmetric Cryptography: Shortcomings

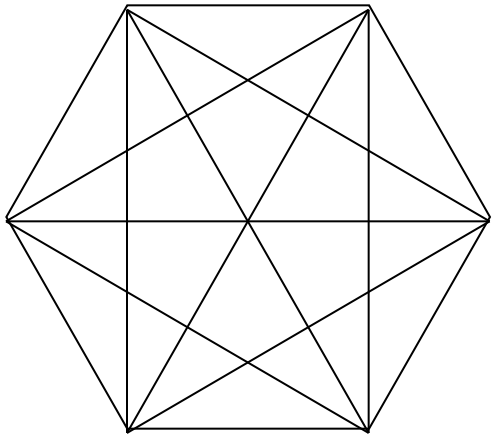
- Symmetric algorithms, e.g., AES or 3DES, are very secure, fast & widespread **but**:
- Key distribution problem: The secret key must be **transported securely**
- Number of keys: In a network, each pair of users requires an individual key

  → *n* users in the network require $\frac{n \cdot (n-1)}{2}$ keys, each user stores ($n-1$) keys

Example:

6 users (nodes)

$\frac{6 \cdot 5}{2} = 15$ keys (edges)



- Alice or Bob can **cheat each other**, because they have identical keys

  Example: Alice can claim that she never ordered a TV on-line from Bob (he could have fabricated her order). To prevent this: “non-repudiation”

Idea behind Asymmetric Cryptography



New Idea:

Use the “good old mailbox” principle:

Everyone can drop a letter

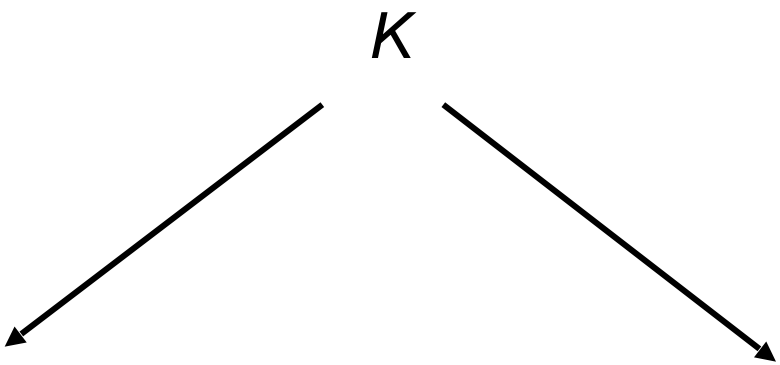
But: Only the owner has the correct key to open the box



1976: first publication of such an algorithm by Whitfield Diffie and Martin Hellman,and also by Ralph Merkle.

Asymmetric (Public-Key) Cryptography

Principle: “Split up” the key

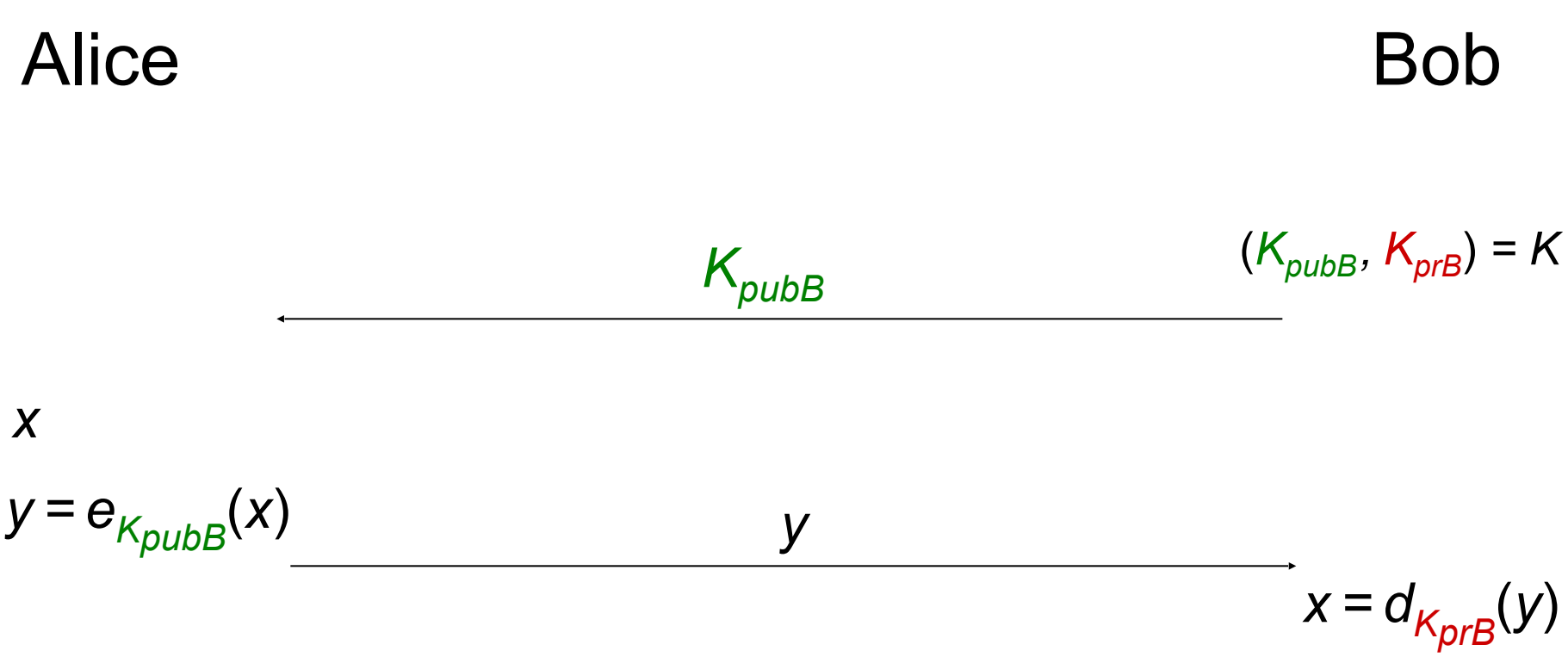


Public Key (K_{pub})
(Encrypt)

Secret Key (K_{pr})
(Decrypt)

→ During the key generation, a key pair K_{pub} and K_{pr} is computed

Basic Protocol for Public-Key Encryption



→ Key Distribution Problem solved *

*) at least for now; public keys need to be authenticated,
cf. Chapter 13 *of Understanding Cryptography*

Security Mechanisms of Public-Key Cryptography

Here are main mechanisms that can be realized with asymmetric cryptography:

- **Key Distribution** (e.g., Diffie-Hellman key exchange, RSA) without a pre-shared secret (key)
- **Nonrepudiation and Digital Signatures** (e.g., RSA, DSA or ECDSA) to provide message integrity
- **Identification**, using challenge-response protocols with digital signatures
- **Encryption** (e.g., RSA / ElGamal)
  Disadvantage: Computationally very intensive
  (1000 times slower than symmetric Algorithms!)

Basic Key Transport Protocol 1/2

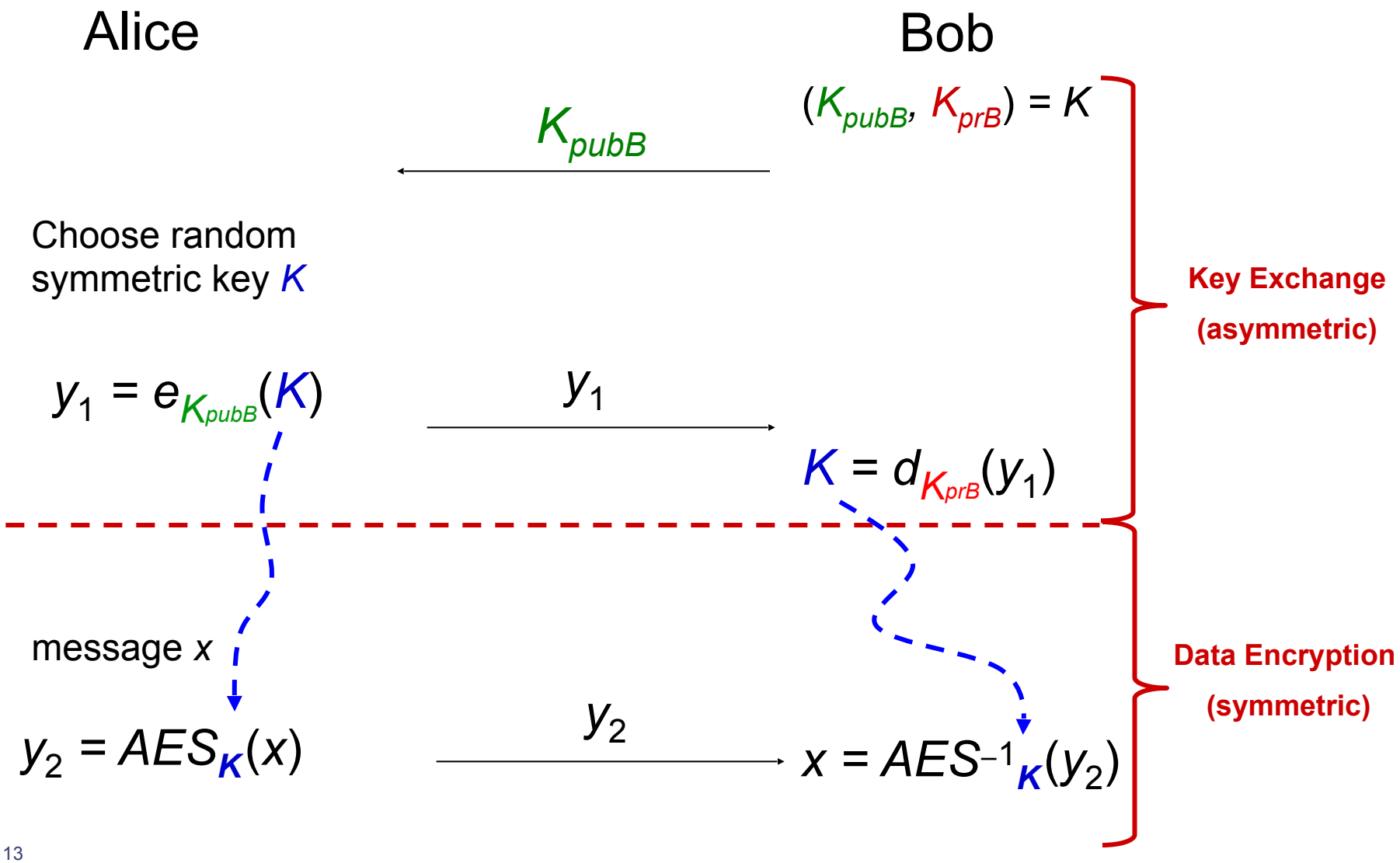
In practice: **Hybrid systems**, incorporating asymmetric and symmetric algorithms

1. **Key exchange** (for symmetric schemes) and **digital signatures** are performed with (slow) **asymmetric** algorithms

2. **Encryption** of data is done using (fast) symmetric ciphers, e.g., **block ciphers or stream ciphers**

Basic Key Transport Protocol 2/2

Example: Hybrid protocol with AES as the symmetric cipher



How to build Public-Key Algorithms

Asymmetric schemes are based on a **"one-way function" *f* ()**:

- Computing $y = f(x)$ is computationally easy
- Computing $x = f^{-1}(y)$ is computationally infeasible

One way functions are based on **mathematically hard problems**.

Three main families:

- **Factoring integers** (RSA, ...):
  Given a composite integer *n*, find its prime factors
  (Multiply two primes: easy)
- **Discrete Logarithm** (Diffie-Hellman, ElGamal, DSA, …):
  Given *a, y* and *m,* find *x* such that $a^x = y \bmod m$
  (Exponentiation a^x : easy)
- **Elliptic Curves (EC)** (ECDH, ECDSA): Generalization of discrete logarithm

Note: The problems are considered mathematically hard, but no proof exists (so far)

Diffie–Hellman Key Exchange: Overview

- Proposed in 1976 by **Whitfield Diffie** and **Martin Hellman**
- **Widely used**, e.g. in Secure Shell (SSH), Transport Layer Security (TLS), and Internet Protocol Security (IPSec)
- The Diffie–Hellman Key Exchange (DHKE) is a key exchange protocol and **not** used for encryption

  (For the purpose of encryption based on the DHKE, ElGamal can be used.)

Diffie–Hellman Key Exchange: Set-up

1. Choose a large prime p
2. Choose an integer $\alpha \in \{2, 3, \ldots, p-2\}$
3. Publish p and α

Diffie–Hellman Key Exchange

Alice		Bob
Choose random private key $k_{prA} = a \in \{1, 2, \ldots, p-1\}$		Choose random private key $k_{prB} = b \in \{1, 2, \ldots, p-1\}$
Compute corresponding public key $k_{pubA} = A = \alpha^a \bmod p$	A →	
	← B	Compute correspondig public key $k_{pubB} = B = \alpha^b \bmod p$
Compute common secret $k_{AB} = B^a = (\alpha^a)^b \bmod p$		Compute common secret $k_{AB} = A^b = (\alpha^b)^a \bmod p$

We can now use the joint key k_{AB} for encryption, e.g., with AES

$y = AES_{k_{AB}}(x)$ → y → $x = AES^{-1}_{k_{AB}}(y)$

Diffie–Hellman Key Exchange: Example

Alice		Bob
Domain parameters $p = 29$, $\alpha = 2$		
Choose random private key $k_{prA} = a = 5$		Choose random private key $k_{prB} = b = 12$
Compute corresponding public key $k_{pubA} = A = 2^5 = 3 \bmod 29$	A →	
	← B	Compute correspondig public key $k_{pubB} = B = 2^{12} = 7 \bmod 29$
Compute common secret $k_{AB} = B^a = 7^5 = 16 \bmod 29$		Compute common secret $k_{AB} = A^b = 3^{12} = 16 \bmod 29$

Proof of correctness:

Alice computes: $B^a = (\alpha^b)^a \bmod p$
Bob computes: $A^b = (\alpha^a)^b \bmod p$

i.e., Alice and Bob obtain the same key k_{AB} !

The Discrete Logarithm Problem

Discrete Logarithm Problem (DLP) in $\boldsymbol{Z}_p^*$

- Given a finite cyclic group $\boldsymbol{Z}_p^*$ of order $p-1$ and a primitive element $\alpha \in \boldsymbol{Z}_p^*$ and another element $\beta \in \boldsymbol{Z}_p^*$
- The DLP is the problem of determining the integer $1 \leq x \leq p-1$ such that

$$\alpha^x \equiv \beta \bmod p$$

- This is called the **discrete logarithm problem (DLP)**

$$x = \log_\alpha \beta \bmod p$$

- Example: Compute x for $5^x \equiv 41 \bmod 47$

The Generalized Discrete Logarithm Problem

- Given a finite cyclic group G with the group operation $\circ$ and cardinality n
- We consider a primitive element $\alpha \in G$ and another element $\beta \in G$
- The discrete logarithm problem is finding the integer x, where $1 \leq x \leq n$, such that:

$$\beta = \underbrace{\alpha \circ \alpha \circ \alpha \circ \ldots \circ \alpha}_{x \text{ times}} = \alpha^x$$

The Generalized Discrete Logarithm Problem

The following discrete logarithm problems have been proposed for use in cryptography

1. The multiplicative group of the prime field $\boldsymbol{Z}_p$ or a subgroup of it. For instance, the classical DHKE uses this group (cf. previous slides), but also Elgamal encryption or the Digital Signature Algorithm (DSA)
2. The cyclic group formed by an elliptic curve
3. The multiplicative group of a Galois field $GF(2^m)$ or a subgroup of it. Schemes such as the DHKE can be realized with them
4. Hyperelliptic curves or algebraic varieties, which can be viewed as generalization of elliptic curves

Remark: The groups 1. and 2. are most often used in practice.

Attacks against the Discrete Logarithm Problem

- Security of many asymmetric primitives is based on the difficulty of computing the DLP in cyclic groups, i.e., compute x for a given α and β such that $\beta = \alpha \circ \alpha \circ \alpha \circ \ldots \circ \alpha = \alpha^x$
- The following algorithms for computing discrete logarithms exist
  - Generic algorithms: Work in any cyclic group
    - Brute-Force Search
    - Shanks' Baby-Step-Giant-Step Method
    - Pollard's Rho (ρ) Method
    - Pohlig-Hellman Method
  - Non-generic Algorithms: Work only in specific groups, in particular in $\boldsymbol{Z}_p$
    - The Index Calculus Method
- Remark: Elliptic curves can only be attacked with generic algorithms which are weaker than non-generic algorithms. Hence, elliptic curves are secure with shorter key lengths than the DLP in prime fields $\boldsymbol{Z}_p$

Attacks against the Discrete Logarithm Problem

Summary of records for computing discrete logarithms in Z_p^*

Decimal digits	Bit length	Date
58	193	1991
68	216	1996
85	282	1998
100	332	1999
120	399	2001
135	448	2006
160	532	2007

In order to prevent attacks that compute the DLP, it is recommended to use primes with a length of at least 1024 bits for schemes such as Diffie-Hellman in Z_p^*

The RSA Cryptosystem

- Ronald **R**ivest, Adi **S**hamir and Leonard **A**dleman proposed the asymmetric RSA cryptosystem in 1977
- Until now, RSA is the most widely use asymmetric cryptosystem although elliptic curve cryptography (ECC) becomes increasingly popular
- RSA is mainly used for two applications
  - Transport of (symmetric) keys
  - Digital signatures

Encryption and Decryption

- RSA operations are done over the integer ring $\boldsymbol{Z}_n$ (i.e., arithmetic modulo n), where $n = p \times q$, with p, q being large primes
- Encryption and decryption are simply exponentiations in the ring

Definition

Given the public key $(n, e) = k_{pub}$ and the private key $d = k_{pr}$,

$$y = e_{k_{pub}}(x) = x^e \mod n$$

$$x = d_{k_{pr}}(y) = y^d \mod n$$

where $x, y \in \boldsymbol{Z}_n$.

Call $e_{k_{pub}}(\,)$ the encryption and $d_{k_{pr}}(\,)$ the decryption operation.

- In practice x, y, n and d are very long integer numbers (≥ 1024 bits)
- The security of the scheme relies on the fact that it is hard to derive the "private exponent" d given the public-key (n, e)

Key Generation

- Like all asymmetric schemes, RSA has set-up phase during which the private and public keys are computed

Algorithm: RSA Key Generation

Output: public key $k_{pub} = (n, e)$ and private key $k_{pr} = d$

1. Choose two large primes p, q
2. Compute $n = p \times q$
3. Compute $\Phi(n) = (p - 1)(q - 1)$
4. Select the public exponent $e \in \{1, 2, \ldots, \Phi(n) - 1\}$ such that $\gcd(e, \Phi(n)) = 1$
5. Compute the private key d such that $d \times e \equiv 1 \pmod{\Phi(n)}$
6. **RETURN** $k_{pub} = (n, e)$, $k_{pr} = d$

Remarks:

- Choosing two large, distinct primes p, q (in Step 1) is non-trivial
- $\gcd(e, \Phi(n)) = 1$ ensures that e has an inverse and, thus, that there is always a private key d

Example: RSA with small numbers

ALICE

Message $x = 4$

BOB

1. Choose $p = 3$ and $q = 11$
2. Compute $n = p \times q = 33$
3. $\Phi(n) = (3 - 1)(11 - 1) = 20$
4. Choose $e = 3$
5. $d \equiv e^{-1} \equiv 7 \bmod 20$

$K_{pub} = (33, 3)$ ←

$y = x^e \equiv 4^3 \equiv 31 \bmod 33$

$y = 31$ →

$y^d = 31^7 \equiv \mathbf{4} = \mathbf{x} \bmod 33$

Implementation Aspects

- How can we implement the modular exponentiation efficiently?

Algorithm: Square-and-Multiply for x^H mod n

Input: Exponent H, base element x, modulus n

Output: $y = x^H \bmod n$

1. Determine binary representation $H = (h_t, h_{t-1}, \ldots, h_0)_2$

2. Initialization: $y = x$

3. **FOR** $i = t-1$ **TO** 0

4. $y = y^2 \bmod n$

5. **IF** $h_i = 1$ **THEN**

6. $y = y \times x \bmod n$

7. **RETURN** y

Example: Square-and-Multiply

Computes x^{26} without modulo reduction

Binary representation of exponent:

$$26 = (1, 1, 0, 1, 0)_2 = (h_4, h_3, h_2, h_1, h_0)_2$$

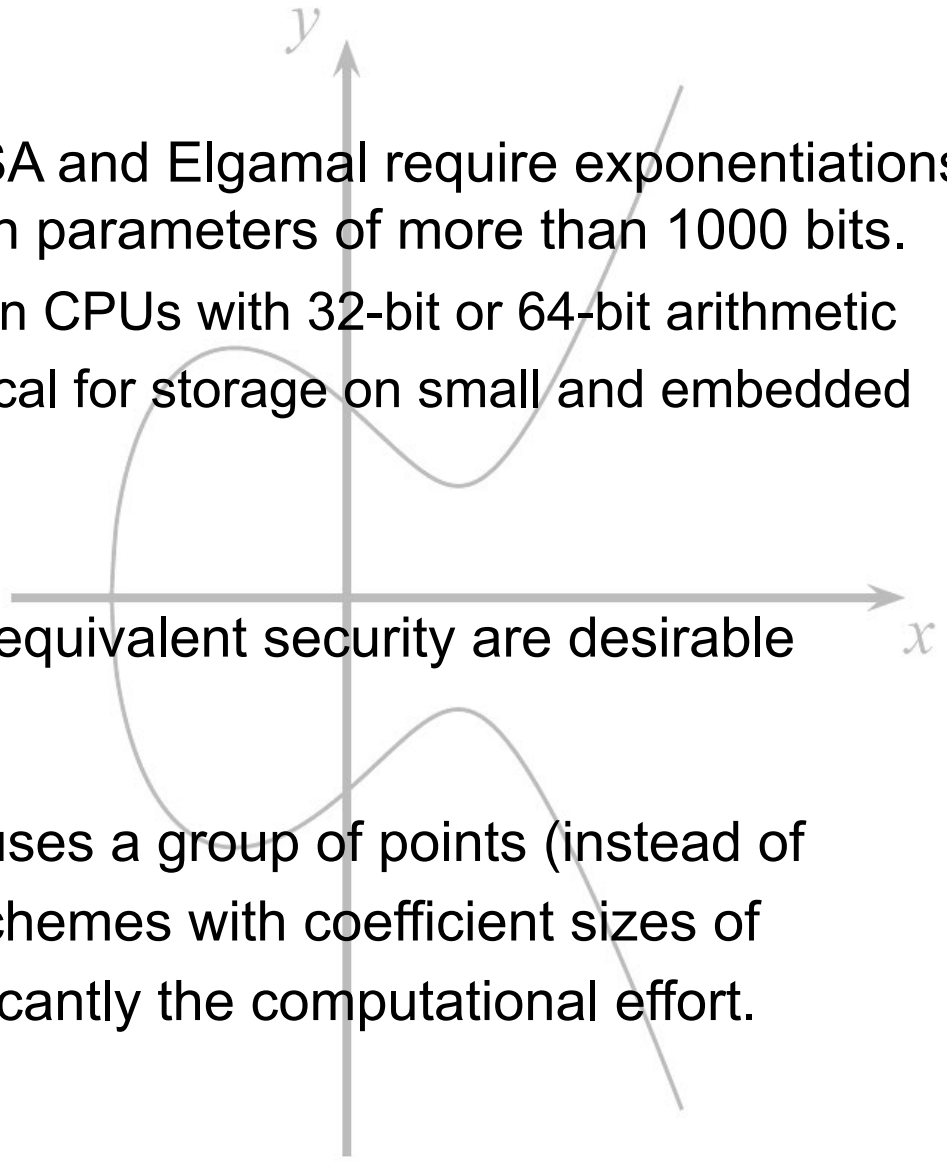
Step		Binary exponent	Op	Comment
1	$x = x^1$	$(1)_2$		Initial setting, h_4 processed
1a	$(x^1)^2 = x^2$	$(10)_2$	SQ	Processing h_3
1b	$x^2 \times x = x^3$	$(11)_2$	MUL	$h_3 = 1$
2a	$(x^3)^2 = x^6$	$(110)_2$	SQ	Processing h_2
2b	-	$(110)_2$	-	$h_0 = 0$
3a	$(x^6)^2 = x^{12}$	$(1100)_2$	SQ	Processing h_1
3b	$x^{12} \times x = x^{13}$	$(1101)_2$	MUL	$h_1 = 1$
4a	$(x^{13})^2 = x^{26}$	$(11010)_2$	SQ	Processing h_0
4b	-	$(11010)_2$	-	$h_0 = 0$

Speed-Up Techniques

- Modular exponentiation is computationally intensive
- Even with the square-and-multiply algorithm, RSA can be quite slow on constrained devices such as smart cards
- Some important tricks:
  - Short public exponent *e*
  - Chinese Remainder Theorem (CRT)
  - Exponentiation with pre-computation (*not covered here*)

Motivation of the Adoption of ECC

- **Problem:**
  Asymmetric schemes like RSA and Elgamal require exponentiations in integer rings and fields with parameters of more than 1000 bits.
  - High computational effort on CPUs with 32-bit or 64-bit arithmetic
  - Large parameter sizes critical for storage on small and embedded systems
- **Motivation:**
  Smaller field sizes providing equivalent security are desirable
- **Solution:**
  Elliptic Curve Cryptography uses a group of points (instead of integers) for cryptographic schemes with coefficient sizes of 160-256 bits, reducing significantly the computational effort.



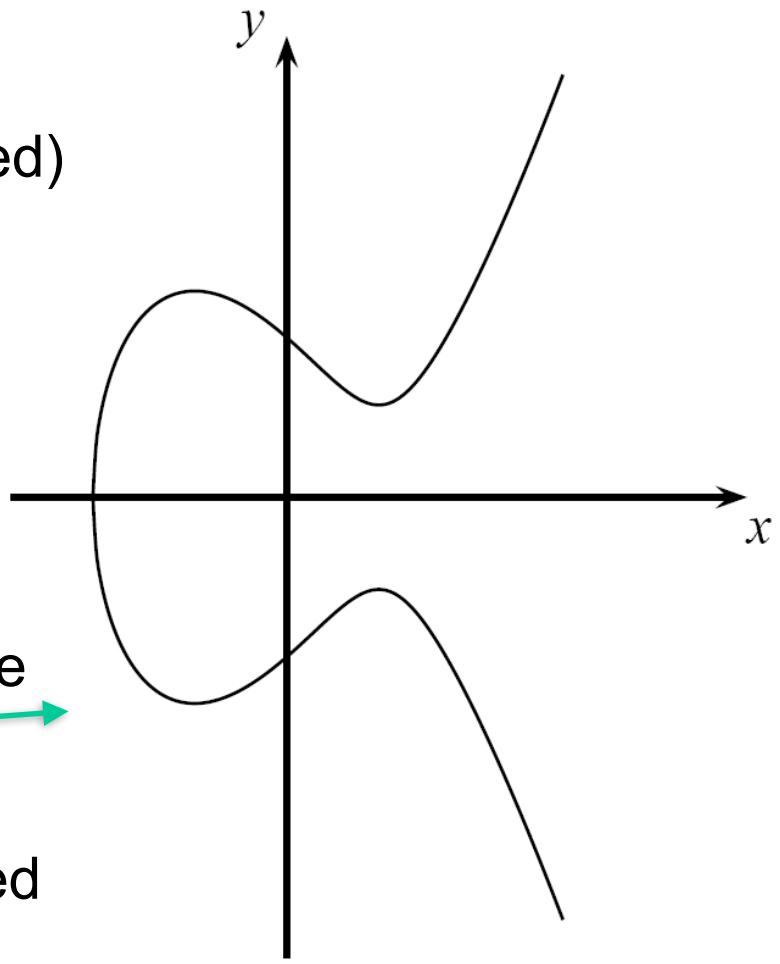
Computations on Elliptic Curves

- Elliptic curves are polynomials that define points based on the (simplified) Weierstraß equation:

$$y^2 = x^3 + ax + b$$

for parameters a, b that specify the exact shape of the curve

- On the real numbers and with parameters $a, b \in \boldsymbol{R}$, an elliptic curve looks like this



Example: $y^2 = x^3 - 3x + 3$ over $\boldsymbol{R}$

- Elliptic curves can not just be defined over the real numbers $\boldsymbol{R}$ but over many other types of finite fields

Computations on Elliptic Curves (ctd.)

- In cryptography, we are interested in elliptic curves module a prime p:

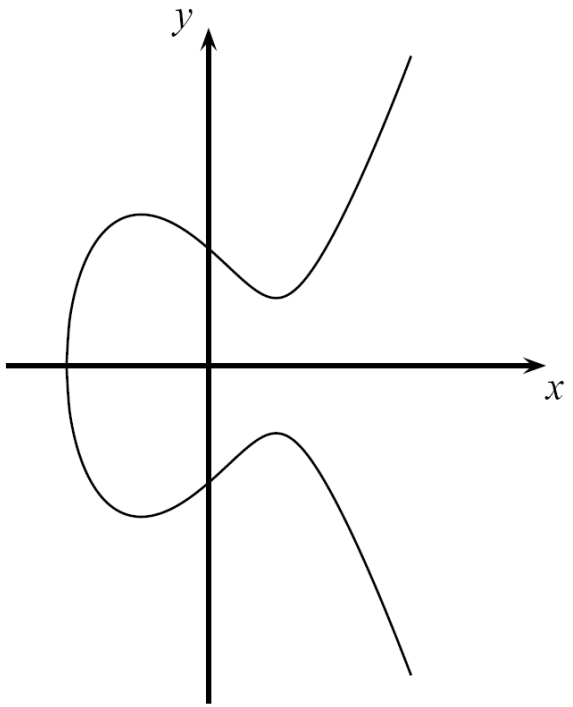
Definition: Elliptic Curves over prime fields

The elliptic curve over $\boldsymbol{Z}_p$, $p > 3$ is the set of all pairs $(x, y) \in \boldsymbol{Z}_p$ which fulfill

$$\boldsymbol{y^2 = x^3 + ax + b \mod p}$$

together with an imaginary point of infinity θ, where $a, b \in \boldsymbol{Z}_p$ and the condition

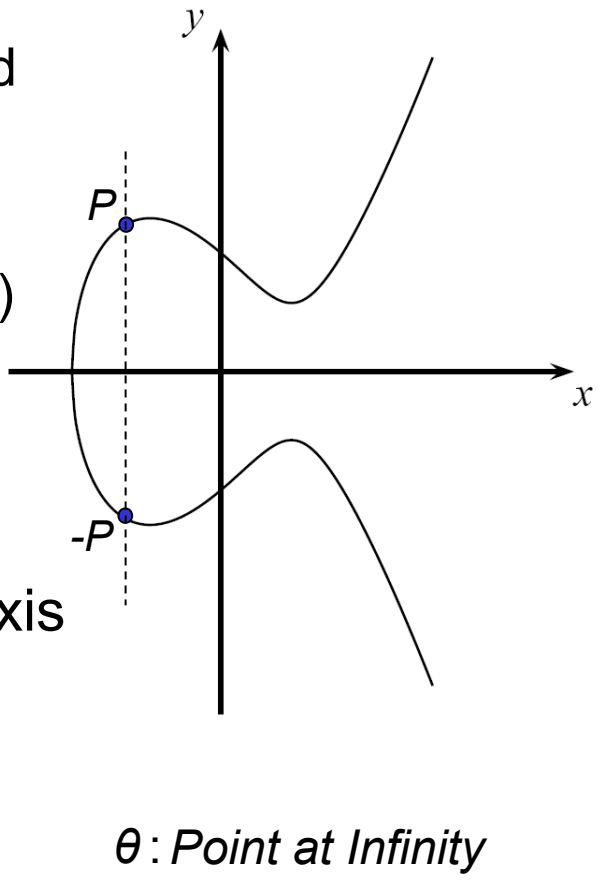
$$\boldsymbol{4a^3 + 27b^2 \neq 0 \mod p}$$



- Note that $\boldsymbol{Z}_p = \{0, 1, \ldots, p-1\}$ is a set of integers with modulo p arithmetic

Computations on Elliptic Curves (ctd.)

- Some special considerations are required to convert elliptic curves into a group of points
  - In any group E, a special element is required to allow for the identity operation, i.e., given $P \in E$: $P + \theta = P = \theta + P$
  - This identity point (which is not on the curve) is additionally added to the group definition
  - This (infinite) identity point is denoted by θ
- Elliptic Curves are symmetric along the x-axis
  - Up to two solutions y and $-y$ exist for each quadratic residue x of the elliptic curve
  - For each point $P = (x, y)$, the inverse or negative point is defined as $-P = (x, -y)$

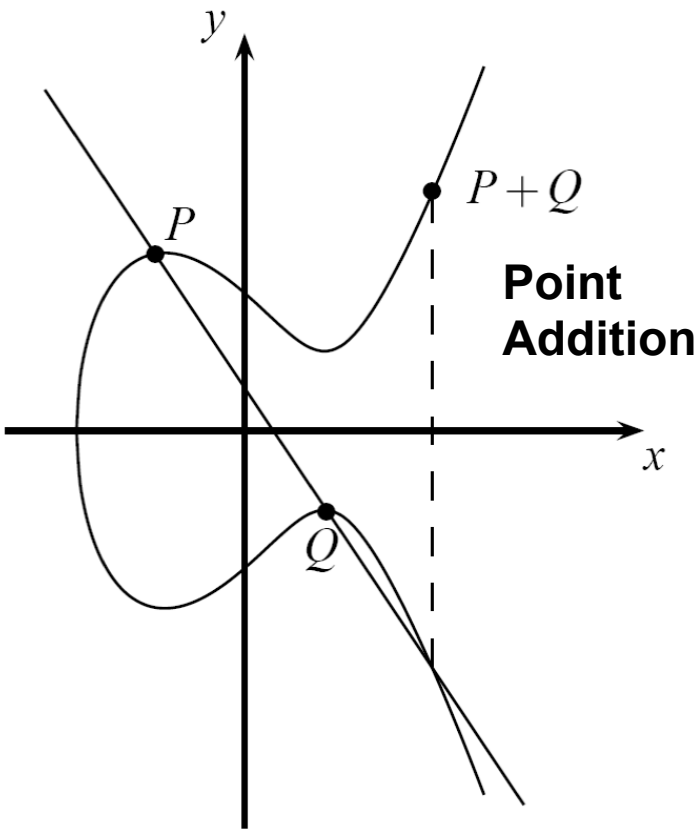


θ : *Point at Infinity*

Computations on Elliptic Curves (ctd.)

- Generating a *group of points* on elliptic curves based on point addition operation $P + Q = R$, i.e., $(x_P, y_P) + (x_Q, y_Q) = (x_R, y_R)$

- Geometric Interpretation of point addition operation
  - Draw straight line through P and Q; if $P = Q$ use tangent line instead
  - Mirror third intersection point of drawn line with the elliptic curve along the x-axis

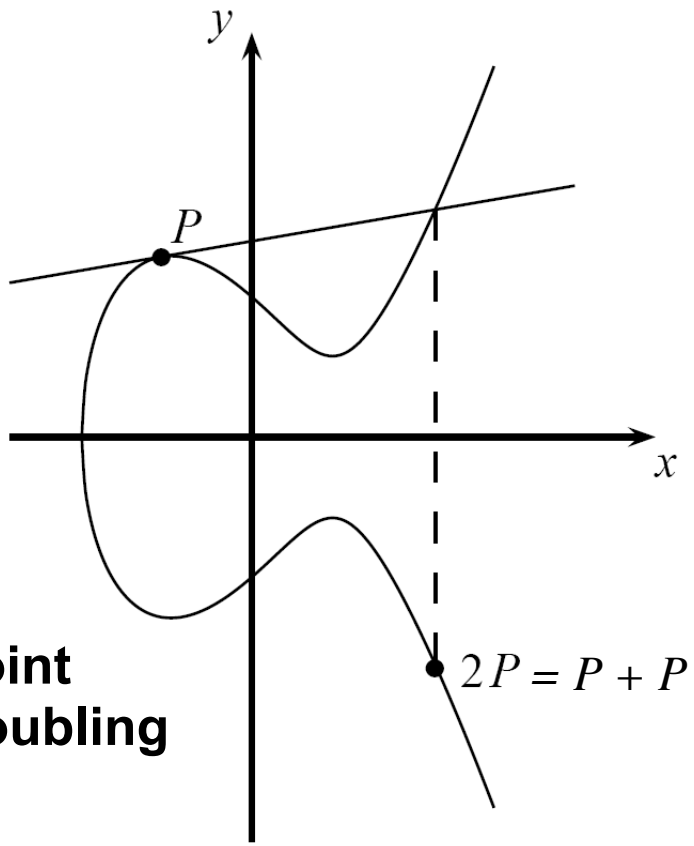


Computations on Elliptic Curves (ctd.)

- Elliptic Curve Point Addition and Doubling Formulas

$x_3 = s^2 - x_1 - x_2 \mod p$ and $y_3 = s(x_1 - x_3) - y_1 \mod p$ where

$$s = \begin{cases} \dfrac{y_2 - y_1}{x_2 - x_1} \mod p, \text{ if } P \neq Q \\ \text{(point addition)} \\ \dfrac{3x_1^2 + a}{2y_1} \mod p, \text{ if } P = Q \\ \text{(point doubling)} \end{cases}$$



Point Doubling

Computations on Elliptic Curves (ctd.)

- **Example**:

Given $E: y^2 = x^3 + 2x + 2 \mod 17$ and point $P = (5, 1)$

Goal: Compute $2P = P + P = (5, 1) + (5, 1) = (x_3, y_3)$

$$s = \frac{3x_1^2 + a}{2y_1} = (2 \cdot 1)^{-1}(3 \cdot 5^2 + 2) = 2^{-1} \cdot 9 \equiv 9 \cdot 9 \equiv 13 \mod 17$$

$$x_3 = s^2 - x_1 - x_2 = 13^2 - 5 - 5 = 159 \equiv 6 \mod 17$$

$$y_3 = s(x_1 - x_3) - y_1 = 13(5 - 6) - 1 = -14 \equiv 3 \mod 17$$

Finally $2P = (5, 1) + (5, 1) = (6, 3)$

Computations on Elliptic Curves (ctd.)

- The points on an elliptic curve and the point at infinity θ form cyclic subgroups

$2P = (5, 1) + (5, 1) = (6, 3)$
$3P = 2P + P = (10, 6)$
$4P = (3, 1)$
$5P = (9, 16)$
$6P = (16, 13)$
$7P = (0, 6)$
$8P = (13, 7)$
$9P = (7, 6)$
$10P = (7, 11)$
$11P = (13, 10)$
$12P = (0, 11)$
$13P = (16, 4)$
$14P = (9, 1)$
$15P = (3, 16)$
$16P = (10, 11)$
$17P = (6, 14)$
$18P = (5, 16)$
$19P = \theta$

This elliptic curve has order $\#E = |E| = 19$
since it contains 19 points in its cyclic group.

Number of Points on an Elliptic Curve

- How many points can be on an arbitrary elliptic curve?
  - Consider the previous example: E: $y^2 = x^3 + 2x + 2$ mod 17 has 19 points
  - However, determining the point count on elliptic curves in general is hard
- Hasse's theorem bounds the number to a restricted interval

> **Definition: Hasse's Theorem:**
> Given an elliptic curve module p, the number of points on the curve is denoted by $\#E$ and is bounded by
>
> $$p + 1 - 2\sqrt{p} \leq \#E \leq p + 1 + 2\sqrt{p}$$

- **Interpretation:** The number of points is "close to" the prime p
- **Example:** To generate a curve with about 2^{160} points, a prime with a length of about 160 bits is required

Elliptic Curve Discrete Logarithm Problem

- Cryptosystems rely on the hardness of the Elliptic Curve Discrete Logarithm Problem (ECDLP)

Definition: Elliptic Curve Discrete Logarithm Problem (ECDLP)

Given a primitive element P and another element T on an elliptic curve E.

The ECDL problem is finding the integer d, where $1 \leq d \leq \#E$ such that

$$\underbrace{P + P + \ldots + P}_{d \text{ times}} = dP = T.$$

- Cryptosystems are based on the idea that d is large and kept secret, and attackers cannot compute it easily
- If d is known, an efficient method to compute the point multiplication dP is required to create a reasonable cryptosystem
  - Known Square-and-Multiply Method can be adapted to Elliptic Curves
  - The method for efficient point multiplication on elliptic curves: Double-and-Add Algorithm

Double-and-Add Algorithm for Point Multiplication

Double-and-Add Algorithm

Input: Elliptic curve E, a point P on E, and a scalar d with bits d_i

Output: $T = dP$

Initialization: $T = P$

Algorithm:

FOR $i = t - 1$ DOWNTO 0

  $T = T + T \bmod n$

  IF $d_i = 1$

    $T = T + P \bmod n$

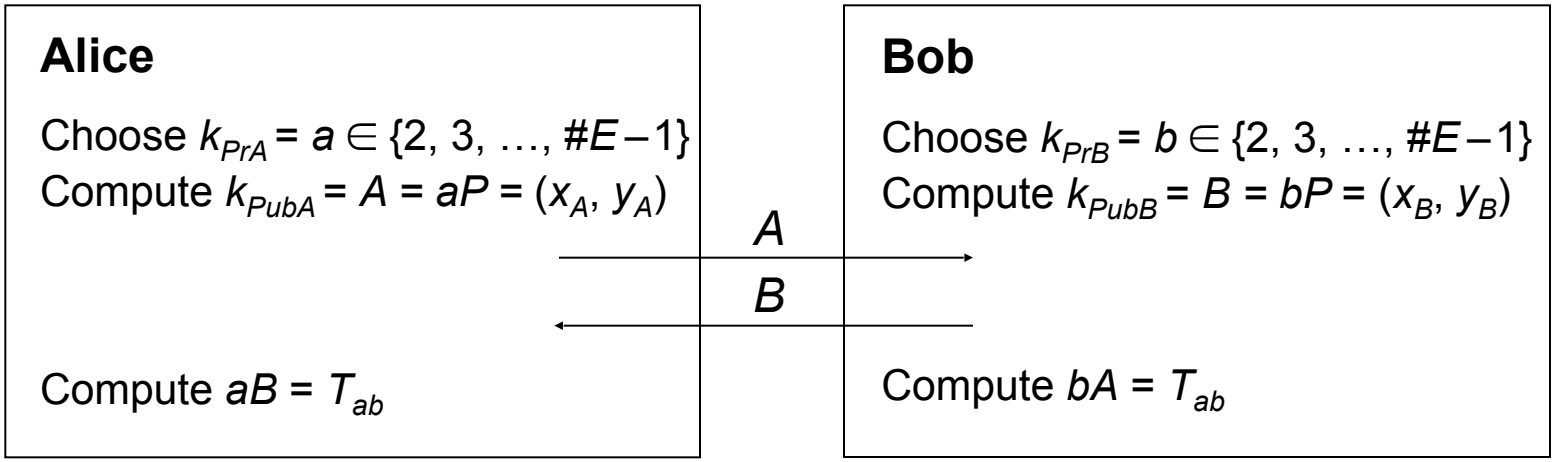
RETURN (T)

Example: $26P = (11010_2)P = (d_4 d_3 d_2 d_1 d_0)_2 P$.

Step		
#0	$P = \mathbf{1}_2 P$	inital setting
#1*a*	$P + P = 2P = \mathbf{10}_2 P$	DOUBLE (bit d_3)
#1*b*	$2P + P = 3P = 10^2 P + 1_2 P = \mathbf{11}_2 P$	ADD (bit $d_3 = 1$)
#2*a*	$3P + 3P = 6P = 2(11_2 P) = \mathbf{110}_2 P$	DOUBLE (bit d_2)
#2*b*		no ADD ($d_2 = 0$)
#3*a*	$6P + 6P = 12P = 2(110_2 P) = \mathbf{1100}_2 P$	DOUBLE (bit d_1)
#3*b*	$12P + P = 13P = 1100_2 P + 1_2 P = \mathbf{1101}_2 P$	ADD (bit d_1=1)
#4*a*	$13P + 13P = 26P = 2(1101_2 P) = \mathbf{11010}_2 P$	DOUBLE (bit d_0)
#4*b*		no ADD ($d_0 = 0$)

The Elliptic Curve Diffie-Hellman Key Exchange (ECDH)

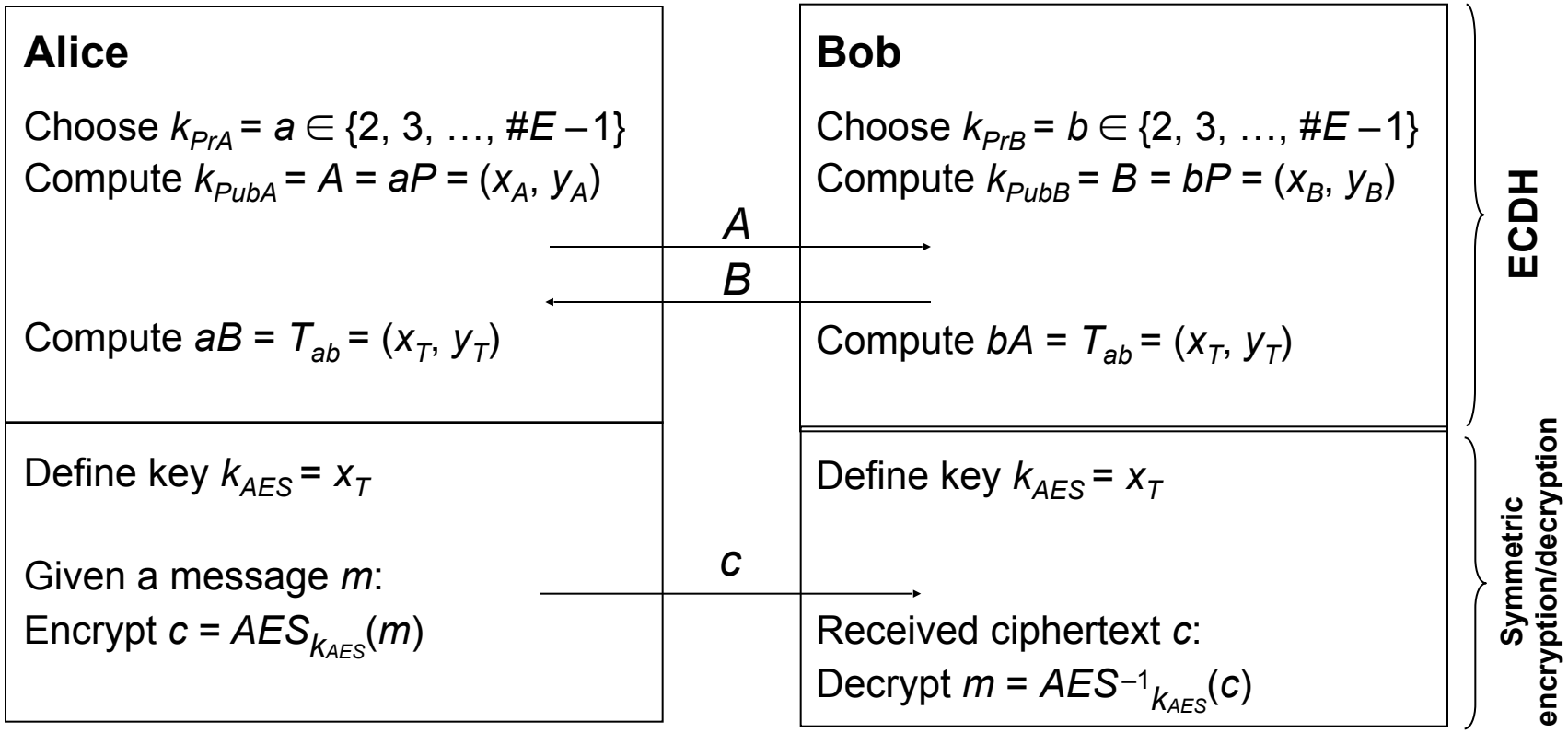
- Given a prime p, a suitable elliptic curve E and a point $P = (x_P, y_P)$
- The Elliptic Curve Diffie-Hellman Key Exchange is defined by the following protocol:



- Joint secret between Alice and Bob: $T_{AB} = (x_{AB}, y_{AB})$
- Proof for correctness:
  - Alice computes $aB = a(bP) = abP$
  - Bob computes $bA = b(aP) = abP$ since group is associative
- ECDH is often used to derive session keys for (symmetric) encryption

The Elliptic Curve Diffie-Hellman Key Exchange (ECDH) (ctd.)

- One of the coordinates of the point T_{AB} (usually the x-coordinate) can be used as session key (often after applying a hash function)

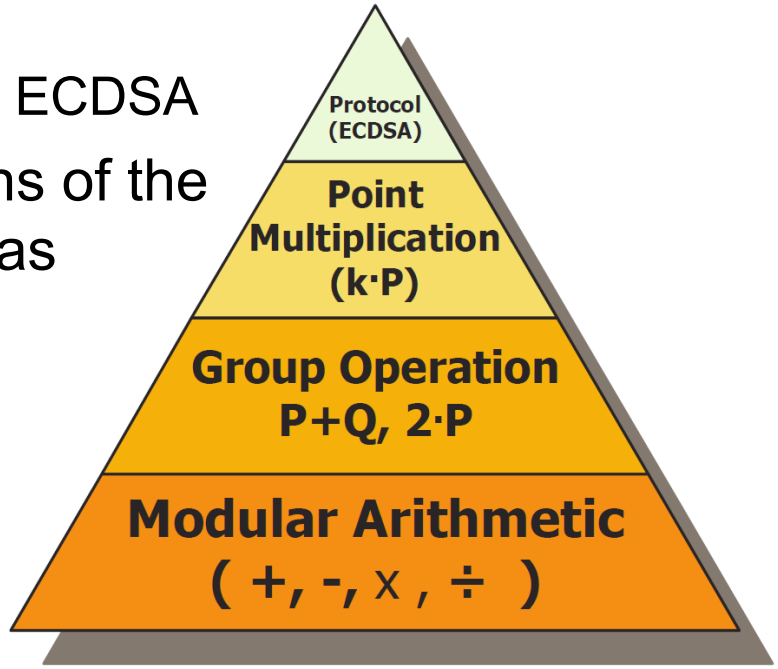


Security Aspects

- Why are parameters significantly smaller for elliptic curves (160-256 bits) than for RSA (1024-3076 bits)?
  - Attacks on groups of elliptic curves are weaker than available factoring algorithms or integer DL attacks
  - Best known attacks on elliptic curves (chosen according to cryptographic criterions) are the Baby-Step Giant-Step and Pollard-Rho method
  - Complexity of these methods: on average, roughly $\sqrt{p}$ steps are required before the ECDLP can be successfully solved
- Implications to practical parameter sizes for elliptic curves:
  - An elliptic curve using a prime p with 160 bit (and roughly 2^{160} points) provides a security of 2^{80} steps that required by an attacker (on average)
  - An elliptic curve using a prime p with 256 bit (roughly 2^{256} points) provides a security of 2^{128} steps on average

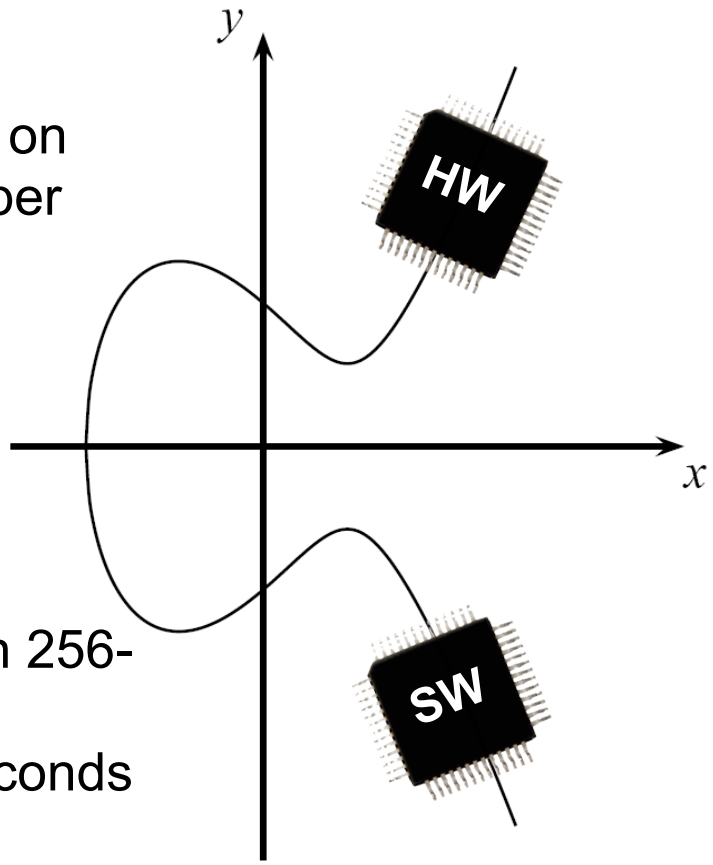
Implementations in Hardware and Software

- Elliptic curve computations are usually regarded as consisting of four layers:
  - Basic modular arithmetic operations are computationally most expensive
  - Group operation implements point doubling and point addition
  - Point multiplication can be implemented using the Double-and-Add method
  - Upper layer protocols like ECDH and ECDSA
- Most efforts should go in optimizations of the modular arithmetic operations, such as
  - Modular addition and subtraction
  - Modular multiplication
  - Modular inversion

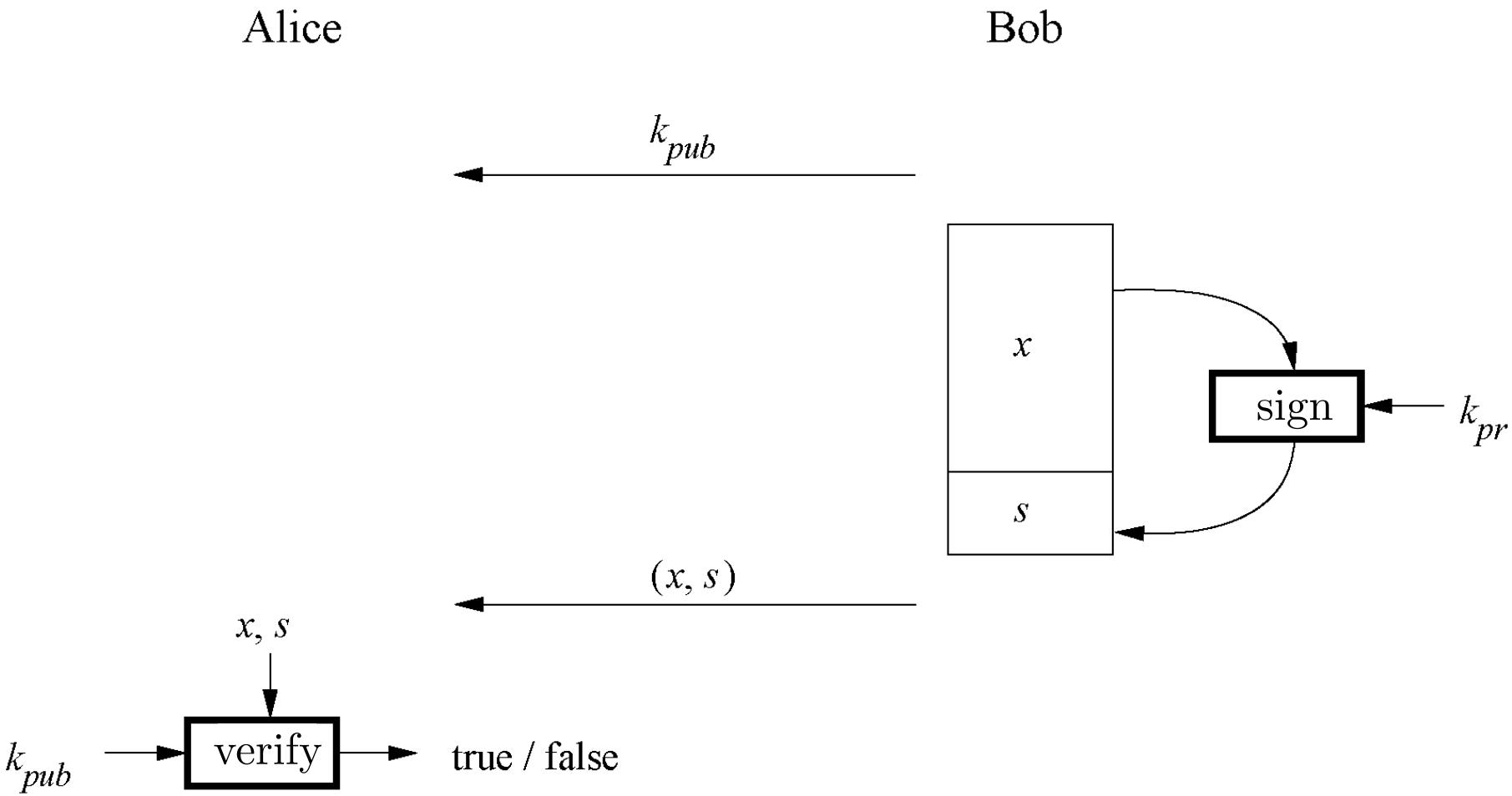


Implementations in Hardware and Software

- Software implementations
  - Optimized 256-bit ECC implementation on 3GHz 64-bit CPU requires about 2 *ms* per point multiplication
  - Less powerful microprocessors (e.g, on SmartCards or cell phones) even take significantly longer (>10 *ms*)
- Hardware implementations
  - High-performance implementations with 256-bit special primes can compute a point multiplication in a few hundred microseconds on reconfigurable hardware
  - Dedicated chips for ECC can compute a point multiplication even in a few ten microseconds



Basic Principle of Digital Signatures



Main Idea of the RSA Signature Scheme

To generate the private and public key:

- Use the same key generation as RSA encryption.

To generate the signature:

- Sign ("encrypt") the message x with the private key

$$s = sig_{K_{pr}}(x) = x^d \bmod n$$

- Append s to message x

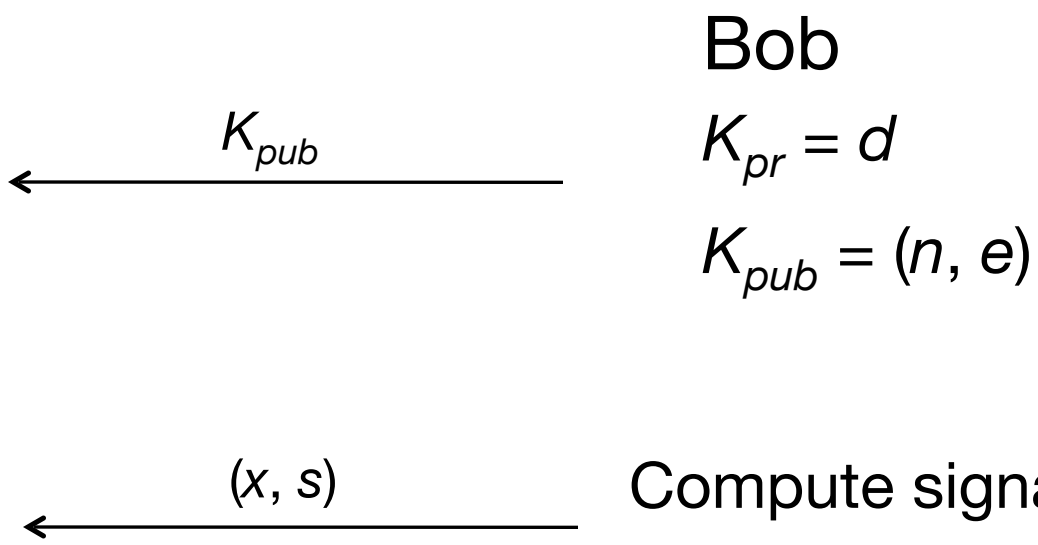
To verify the signature:

- Verify ("decrypt") the signature with the public key

$$x' = ver_{K_{pub}}(s) = s^e \bmod n$$

- If $x = x'$, the signature is valid

The RSA Signature Protocol



Verify signature:

$x' \equiv s^e \bmod n$

If $x' \not\equiv x \rightarrow$ valid signature

If $x' = x \rightarrow$ invalid signature

Security and Performance of the RSA Signature Scheme

Security:

The same constrains as RSA encryption: *n* needs to be at least 1024 bits to provide a security level of 80 bits.

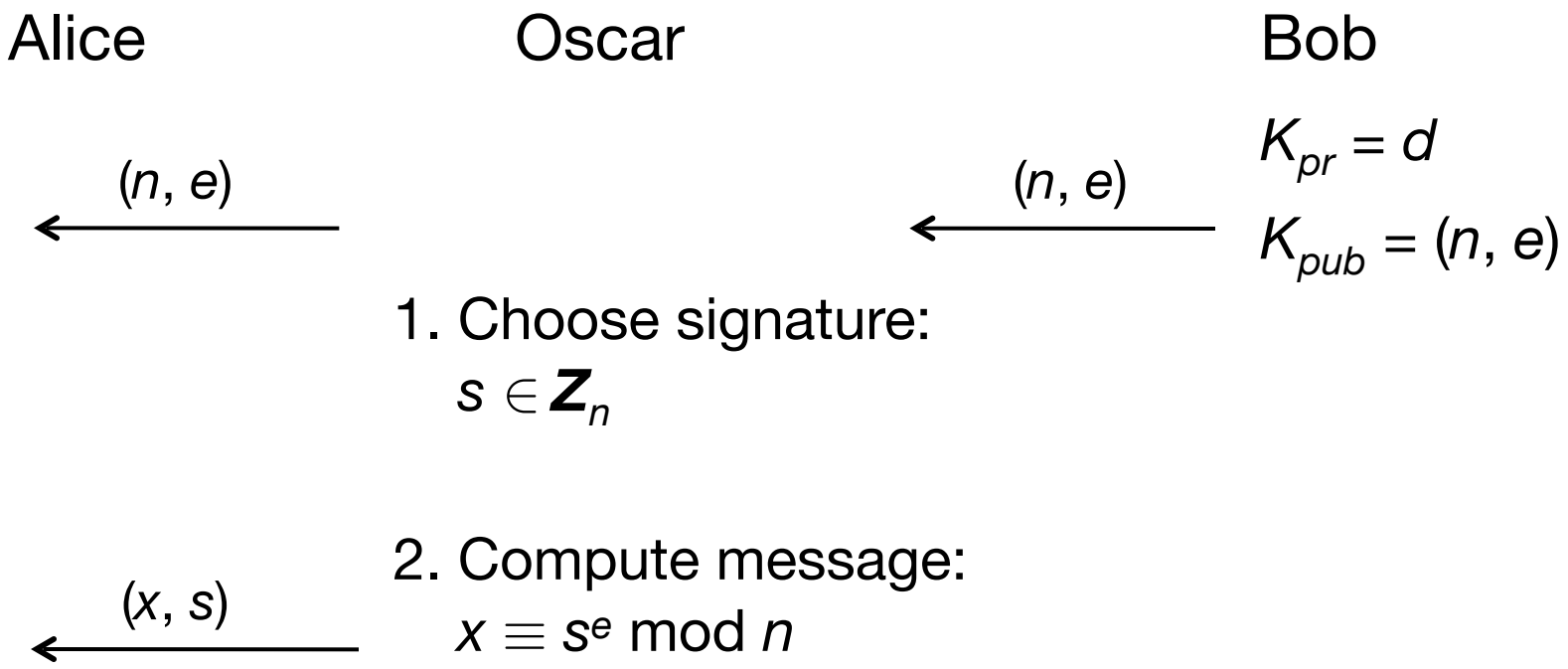
$\Rightarrow$ The signature, consisting of *s*, needs to be at least 1024 bits long.

Performance:

The signing process is an exponentiation with the private key and the verification process as an exponentiation with the public key *e*.

$\Rightarrow$ Signature verification is very efficient as a small number can be chosen for the public key.

Existential Forgery Attack against RSA Digital Signature



Verification:

$s^e \equiv x' \bmod n$

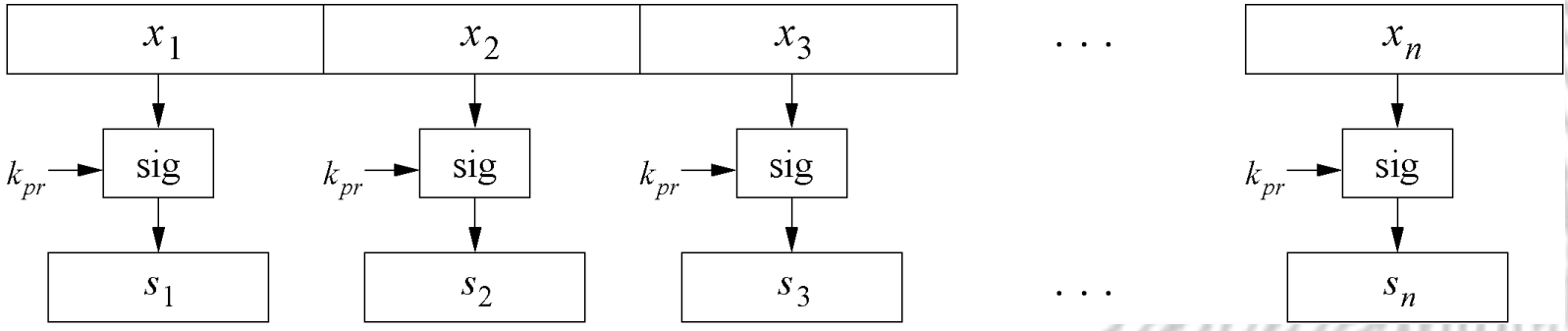
since $s^e = (x^d)^e \equiv x \bmod n$

→ Signature is valid

Motivation

Problem:

Naive signing of long messages generates a signature of same length.



- Three Problems
  - Computational overhead
  - Message overhead
  - Security limitations
- For more info see Section 11.1 in "*Understanding Cryptography*".

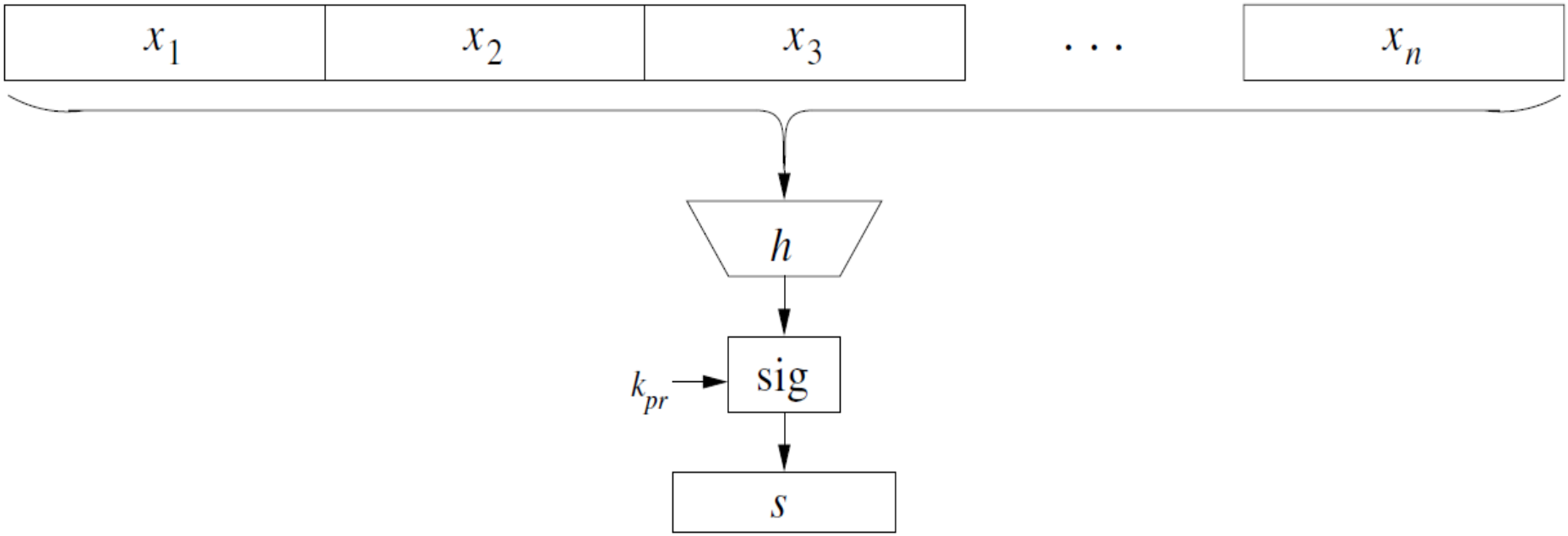
Solution:

- Instead of signing the whole message, sign only a digest (=hash)
- Also secure, but much faster

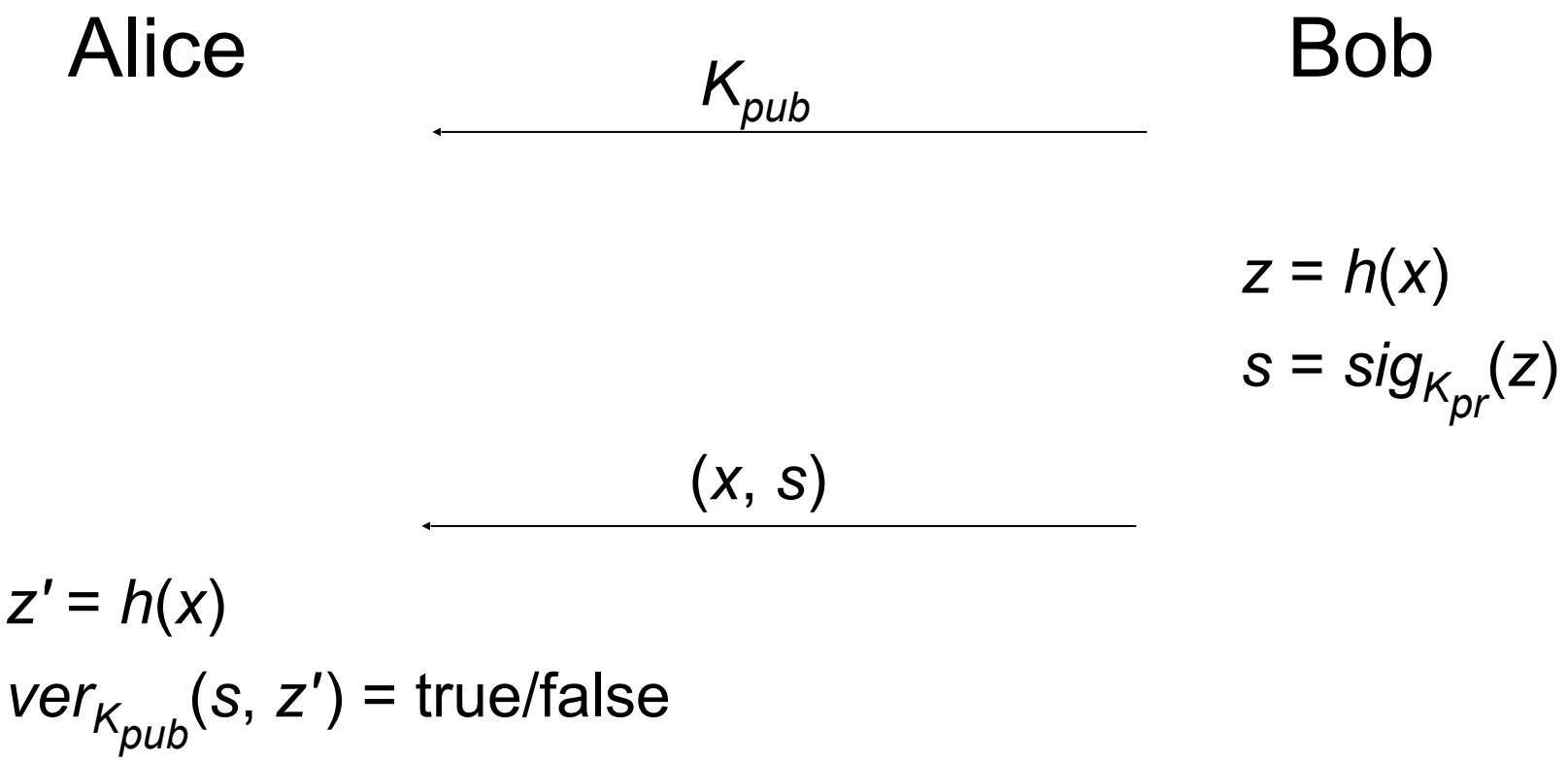
Needed:

Hash Functions

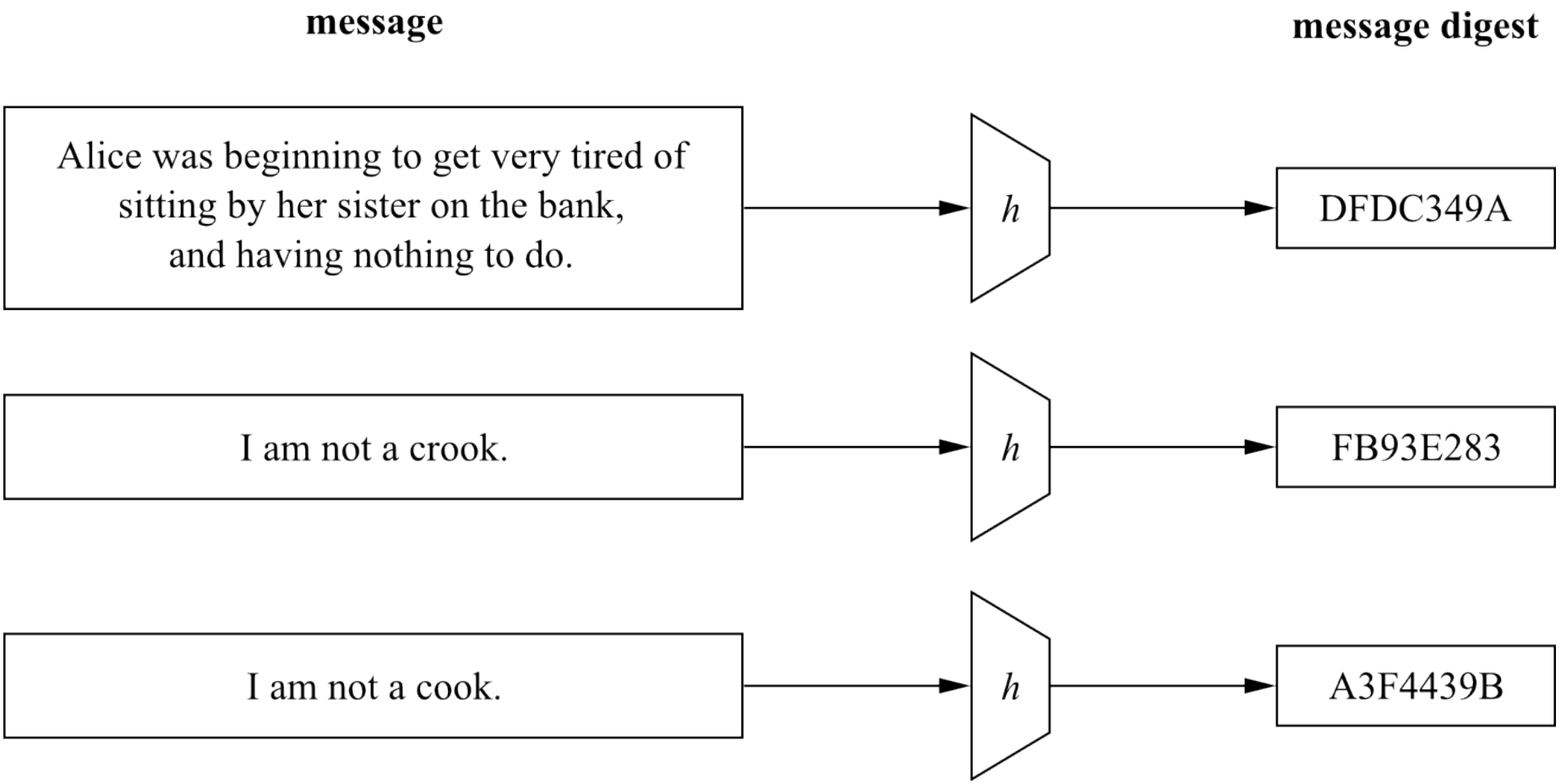
Digital Signature with a Hash Function



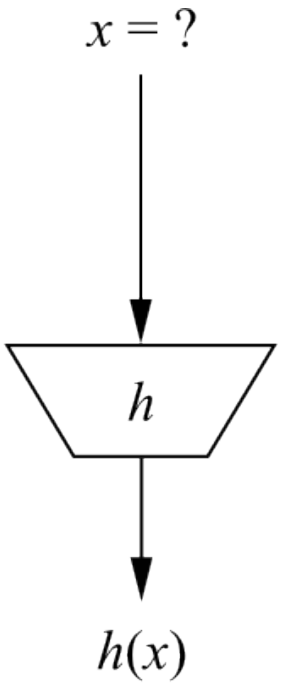
Basic Protocol for Digital Signatures with a Hash Function



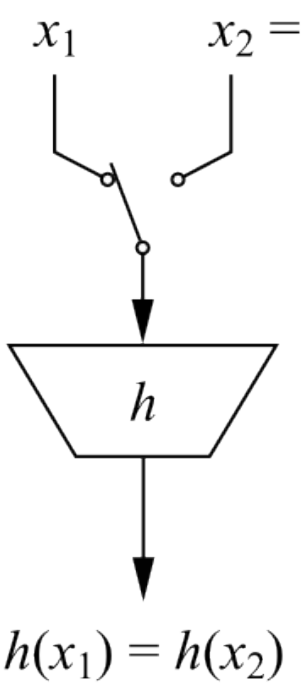
Principal input – output behavior of hash functions



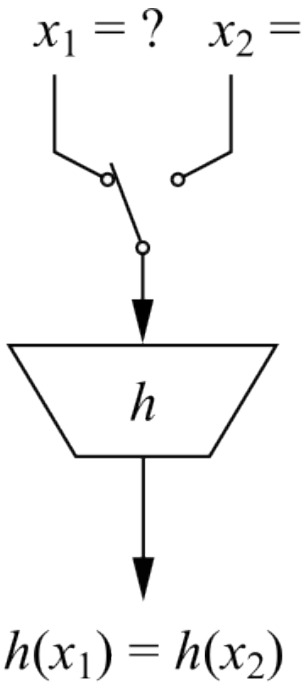
The three security properties of hash functions



preimage resistance



second preimage resistance



collision resistance

Hash Function: Security Properties

- **Pre-image resistance:**
  For a given output z, it is computationally infeasible to find any input x such that $h(x) = z$, i.e., $h(x)$ is one-way

- **Second pre-image resistance:**
  Given x_1, and thus $h(x_1)$, it is computationally infeasible to find any x_2 such that $h(x_1) = h(x_2)$

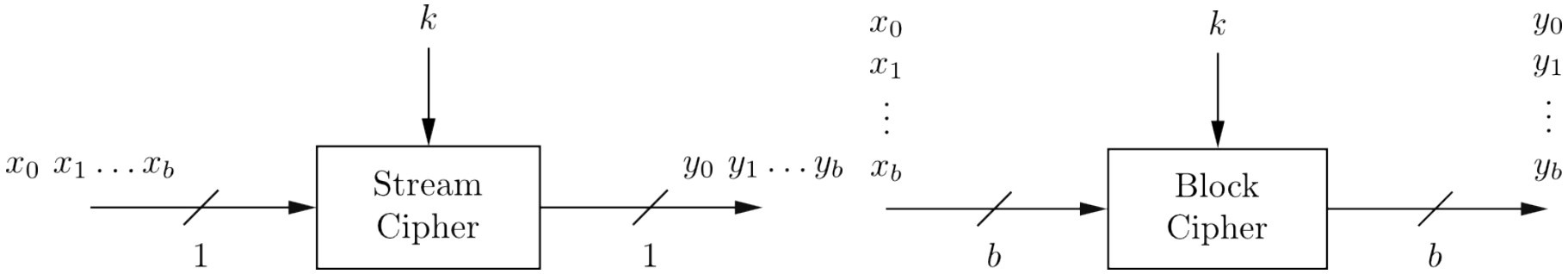
- **Collision resistance:**
  It is computationally infeasible to find any pairs $x_1 \neq x_2$ such that $h(x_1) = h(x_2)$

Hash Function : Security

It turns out that collision resistance causes most problems

- How hard is it to find a collision with a probability of 0.5?
- Related Problem: How many people are needed such that two of them have the same birthday with a probability of 0.5?
- Far fewer than 365/2 = 182.5! This is called the birthday paradox (Search takes ≈ $\sqrt{2^n}$ steps).
- For more information see Chapter 11.2.3 in *Understanding Cryptography*.
- To deal with this paradox, hash functions need a output size of at least 160 bits.

Stream Cipher vs. Block Cipher

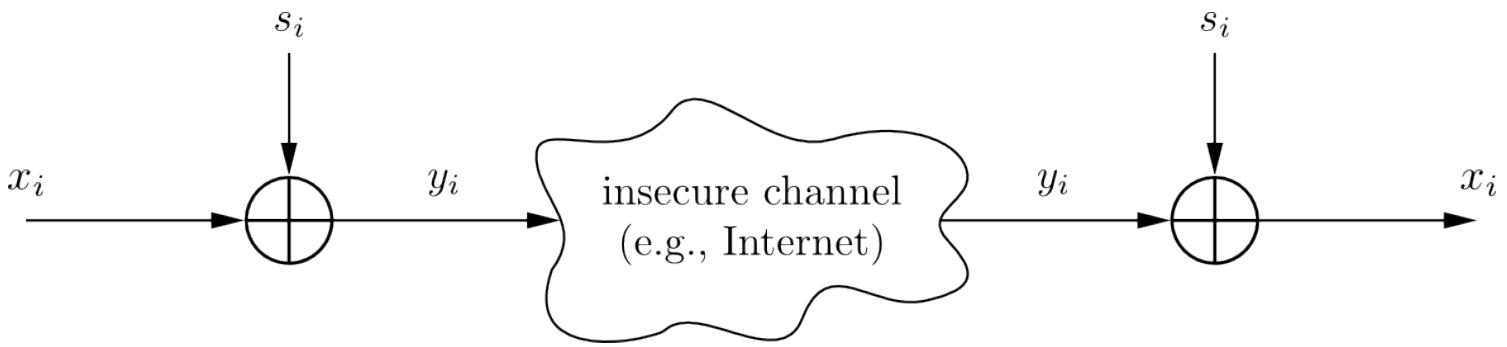


- **Stream Ciphers**
  - Encrypt bits individually
  - Usually small and fast
    → common in embedded devices (e.g., A5/1 for GSM phones)

- **Block Ciphers:**
  - Always encrypt a full block (several bits)
  - Are common for Internet applications

Encryption and Decryption with Stream Ciphers

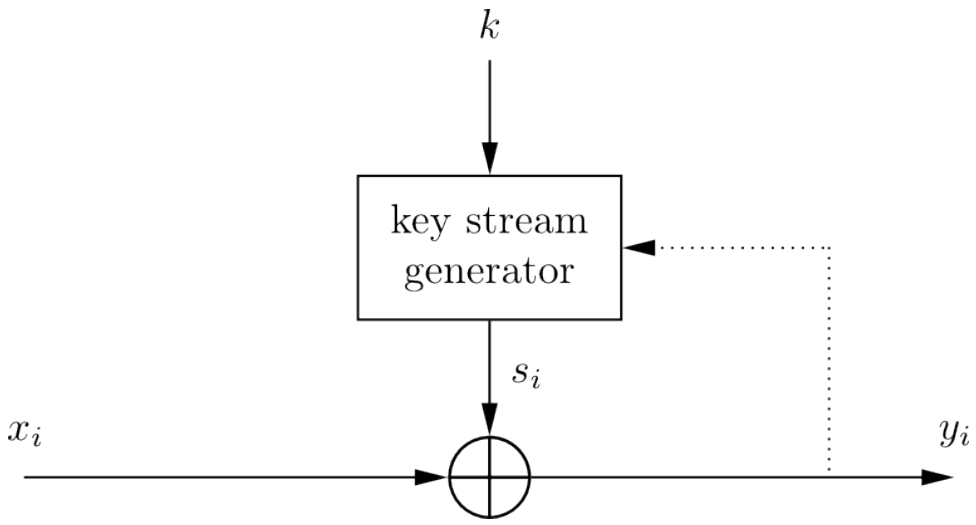
Plaintext x_i, ciphertext y_i, and key stream s_i consist of individual bits



- Encryption and decryption are simple additions modulo 2 (aka XOR)
- Encryption and decryption are the same functions

- **Encryption:** $y_i = e_{s_i}(x_i) = x_i + s_i \bmod 2$ $\quad x_i, y_i, s_i \in \{0, 1\}$
- **Decryption:** $x_i = e_{s_i}(y_i) = y_i + s_i \bmod 2$

Synchronous vs. Asynchronous Stream Cipher



- Security of stream cipher depends entirely on the key stream s_i :
  - Should be **random** , i.e., $\Pr(s_i = 0) = \Pr(s_i = 1) = 0.5$
  - Must be **reproducible** by sender and receiver
- **Synchronous Stream Cipher**
  - Key stream depend only on the key (and possibly an initialization vector IV)
- **Asynchronous Stream Ciphers**
  - Key stream depends also on the ciphertext (dotted feedback enabled)

Why is Modulo 2 Addition a Good Encryption Function?

- Modulo 2 addition is equivalent to XOR operation
- For perfectly random key stream s_i , each ciphertext output bit has a 50% chance to be 0 or 1
  → Good statistic property for ciphertext
- Inverting XOR is simple, since it is the same XOR operation

x_i	s_i	y_i
0	0	0
0	1	1
1	0	1
1	1	0

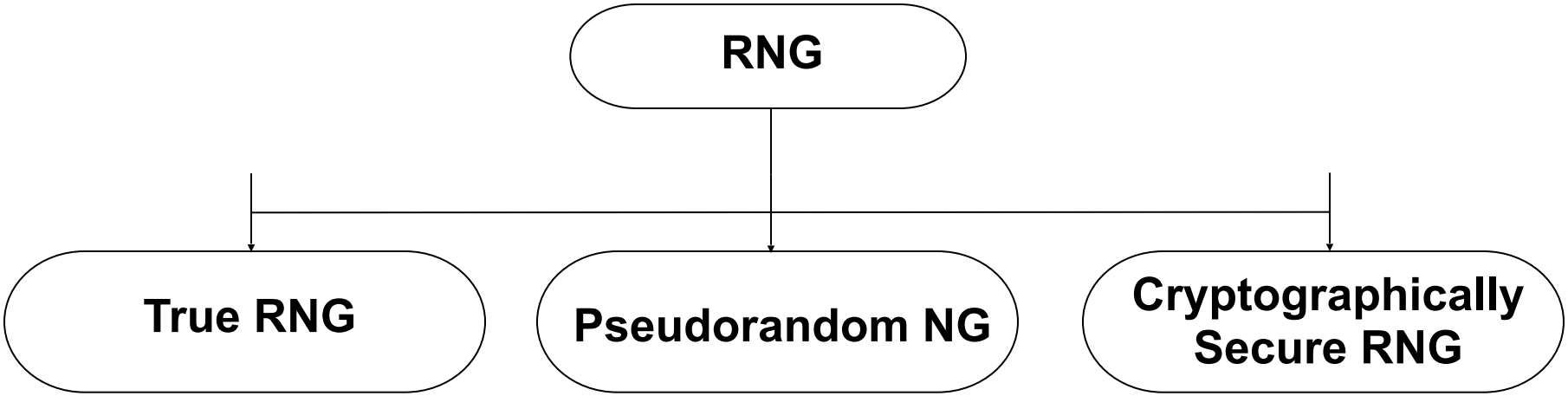
Stream Cipher: Throughput

Performance comparison of symmetric ciphers (Pentium4):

Cipher	Key length	Mbit/s
DES	56	36.95
3DES	112	13.32
AES	128	51.19
RC4 (stream cipher)	(choosable)	211.34

Source: Zhao et al., Anatomy and Performance of SSL Processing, ISPASS 2005

Random number generators (RNGs)

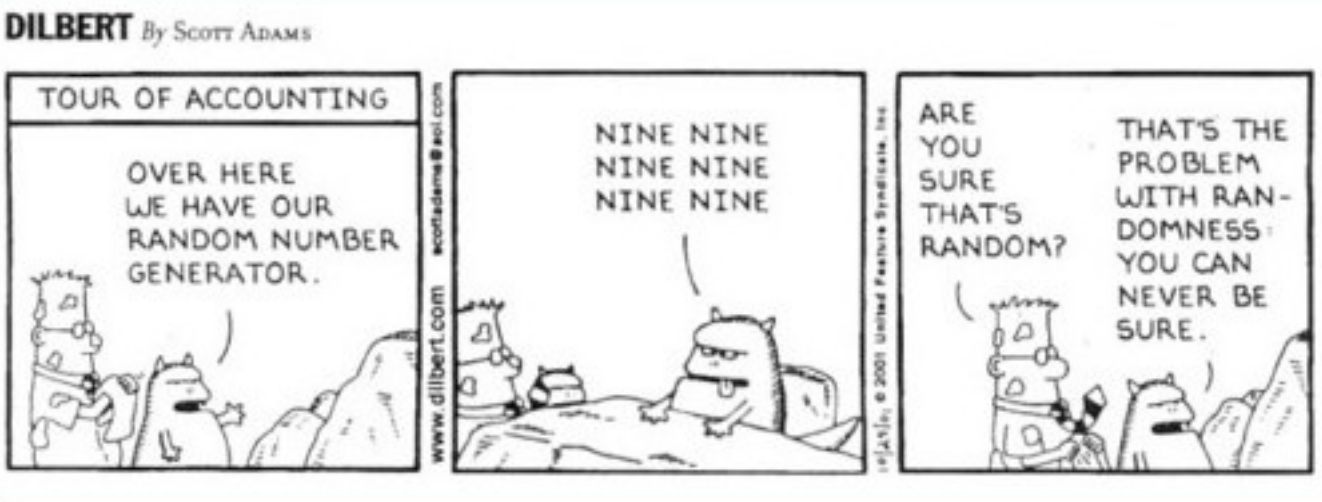


True Random Number Generators (TRNGs)

- Based on physical random processes: coin flipping, dice rolling, semiconductor noise, radioactive decay, mouse movement, clock jitter of digital circuits
- Output stream s_i should have good statistical properties:

  $\Pr(s_i = 0) = \Pr(s_i = 1) = 50\%$ (often achieved by post-processing)
- Output can neither be predicted nor be reproduced

  Typically used for generation of keys, nonces (used only-once values) and for many other purposes



Pseudorandom Number Generator (PRNG)

- Generate sequences from initial seed value
- Typically, output stream has good statistical properties
- Output can be reproduced and can be predicted
- Often computed in a recursive way:

$$s_0 = seed$$

$$s_{i+1} = f(s_i, s_{i-1}, \ldots, s_{i-t})$$

Example: *rand*() *function* in ANSI C:

$$s_0 = 12345$$

$$s_{i+1} = 1103515245 s_i + 12345 \mod 2^{31}$$

Most PRNGs have bad cryptographic properties!

Cryptanalyzing a Simple PRNG

Simple PRNG: **Linear Congruential Generator**

$$S_0 = seed$$

$$S_{i+1} = AS_i + B \mod m$$

Assume

- unknown A, B and S_0 as key
- size of A, B and S_i to be 100 bits
- 300 bits of output are known, i.e. S_1, S_2 and S_3

Solving

$$S_2 \equiv AS_1 + B \pmod m$$

$$S_3 \equiv AS_2 + B \pmod m$$

…directly reveals A and B. All S_i can be computed easily!

Bad cryptographic properties due to the linearity of most PRNGs

Cryptographically Secure Pseudorandom Number Generator (CSPRNG)

- Special PRNG with additional property:
  - Output must be **unpredictable**
  - More precisely: Given n consecutive bits of output s_i, the following output bits s_{n+1} cannot be predicted (in polynomial time)
- Needed in cryptography, in particular for stream ciphers
- Remark: There are almost no other applications that need unpredictability, whereas many, many (technical) systems need PRNGs

One-Time Pad (OTP)

Unconditionally secure cryptosystem:

- A cryptosystem is *unconditionally secure* if it cannot be broken even with *infinite* computational resources

One-Time Pad

- A cryptosystem developed by Mauborgne that is based on Vernam's stream cipher
- Properties:

  Let the plaintext, ciphertext and key consist of individual bits $x_i, y_i, k_i \in \{0,1\}$

Encryption:	$e_{k_i}(x_i) = x_i \oplus k_i$
Decryption:	$d_{k_i}(y_i) = y_i \oplus k_i$

OTP is unconditionally secure if and only if the key k_i is used once!

One-Time Pad (OTP)

Unconditionally secure cryptosystem:

$$y_0 = x_0 \oplus k_0$$
$$y_1 = x_1 \oplus k_1$$
$$\vdots$$

Every equation is a linear equation with two unknowns

$\Rightarrow$ For every y_i, $x_i = 0$ and $x_i = 1$ are equiprobable!

$\Rightarrow$ This is true iff k_0, k_1, ... are independent, i.e., all k_i have to be generated truly random

$\Rightarrow$ It can be shown that this systems can *provably* not be solved

Disadvantage: For almost all applications the OTP is **impractical** since the key must be as long as the message! (Imagine you have to encrypt a 1GByte email attachment.)

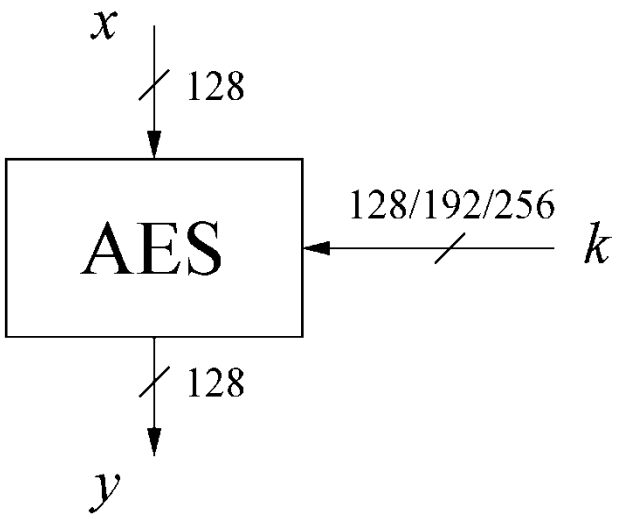
Advanced Encryption Standard

- AES is the most widely used symmetric cipher today
- The algorithm for AES was chosen by the US *National Institute of Standards and Technology* (NIST) in a multi-year selection process
- The requirements for all AES candidate submissions were:
  - Block cipher with **128-bit block size**
  - **Three supported key lengths**: 128, 192 and 256 bits
  - Security relative to other submitted algorithms
  - **Efficiency** in software and hardware

Chronology of the AES Selection

- The need for a new block cipher announced by NIST in January, 1997
- 15 candidates algorithms accepted in August, 1998
- 5 finalists announced in August, 1999:
  - *Rijndael* – J. Daemen & V. Rijmen
  - *Serpent* – Eli Biham et al.
  - *Twofish* – B. Schneier et al.
  - *RC6* – RSA Laboratories
  - *Mars* – IBM Corporation
- In October 2000, *Rijndael* was chosen as the AES
- AES was formally approved as a US federal standard in November 2001

AES: Overview

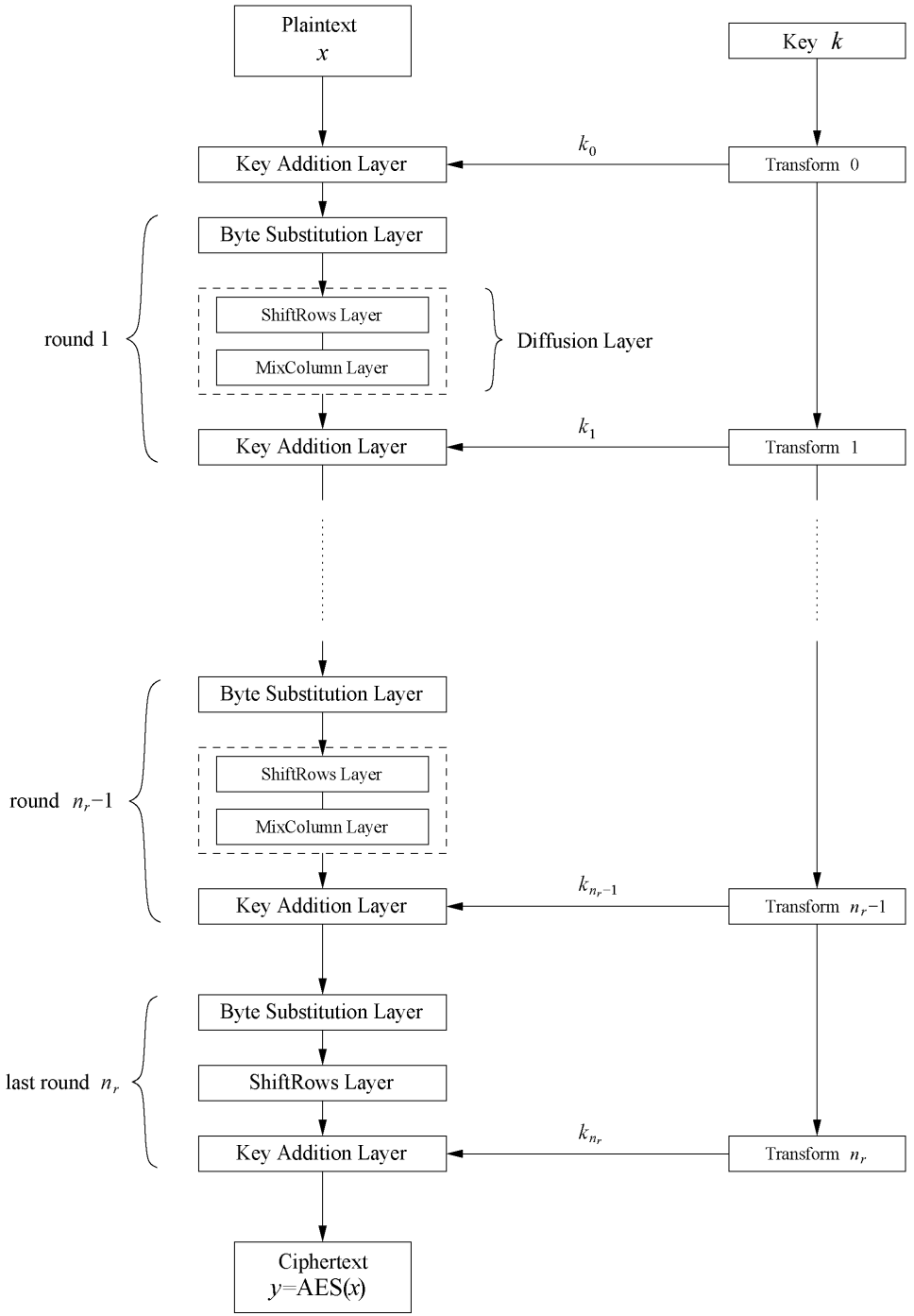


The number of rounds depends on the chosen key length:

Key length (bits)	Number of rounds
128	10
192	12
256	14

AES: Overview

- Iterated cipher with 10/12/14 rounds
- Each round consists of “Layers”



Internal Structure of AES

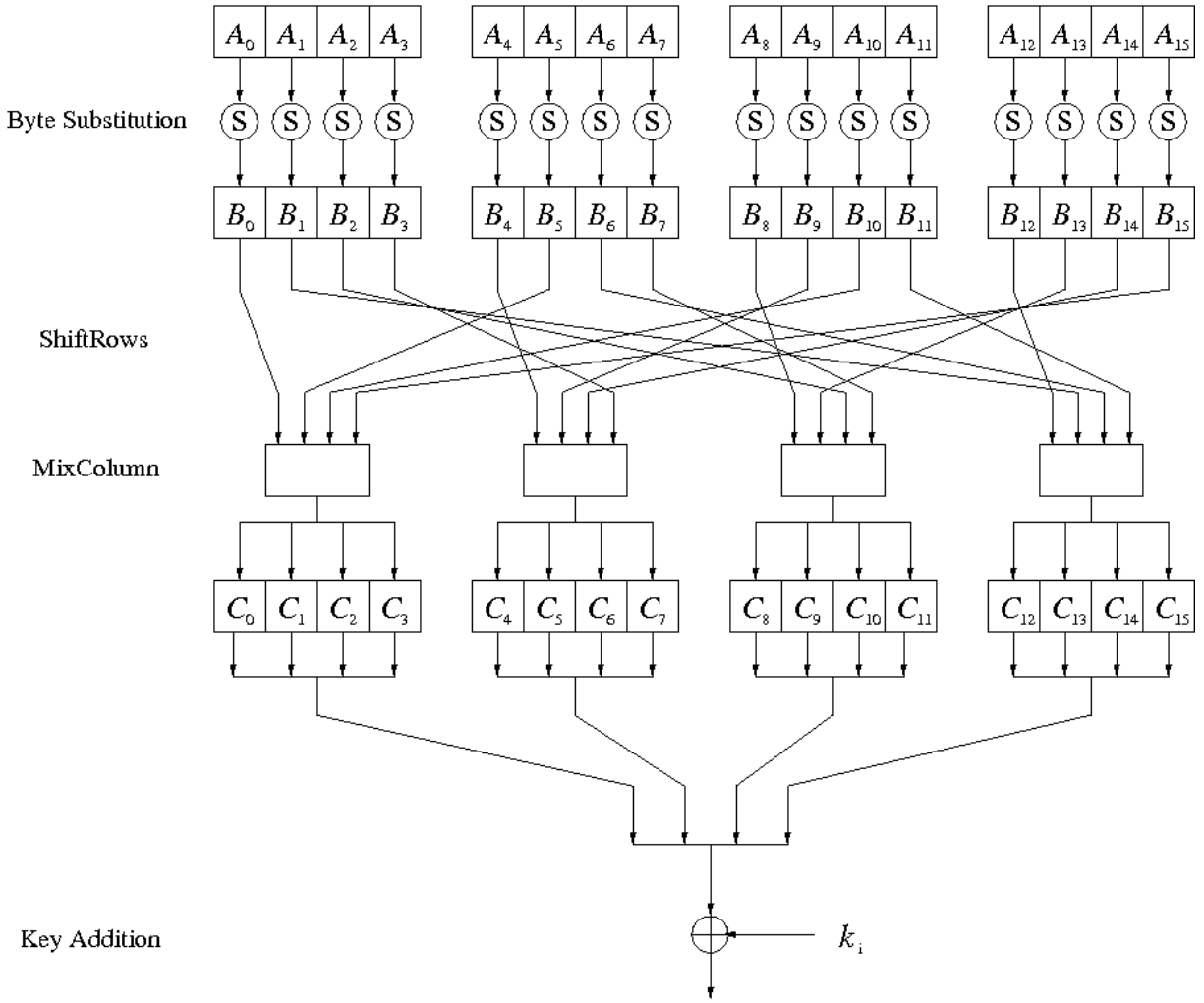
- AES is a byte-oriented cipher
- The state A (i.e., the 128-bit data path) can be arranged in a 4×4 matrix:

A_0	A_4	A_8	A_{12}
A_1	A_5	A_9	A_{13}
A_2	A_6	A_{10}	A_{14}
A_3	A_7	A_{11}	A_{15}

with $A_0, \ldots, A_{15}$ denoting the 16-byte input of AES

Internal Structure of AES

- Round function for rounds 1, 2, …, n_{r-1}:



- Note: In the last round, the MixColumn transformation is omitted

Byte Substitution Layer

- The Byte Substitution layer consists of 16 **S-Boxes** with the following properties:

  The S-Boxes are

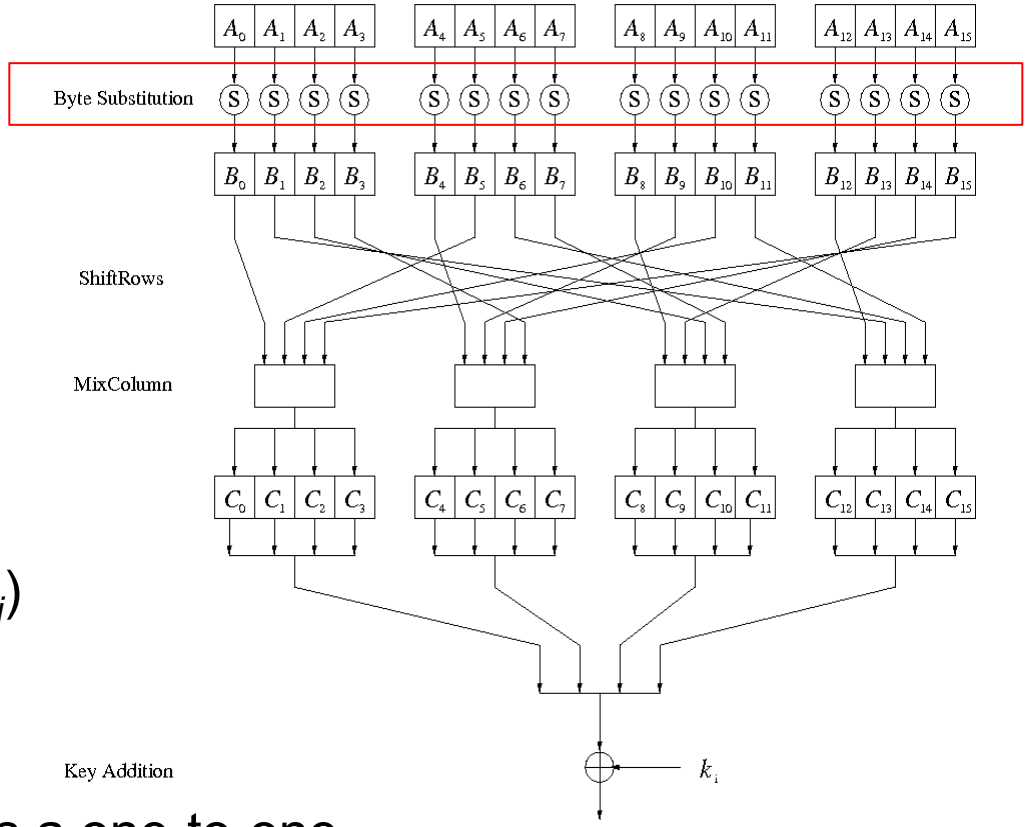
  - **identical**
  - the only **nonlinear** operation of AES, i.e.,

    $\text{ByteSub}(A_i) + \text{ByteSub}(A_j)$

    $\neq \text{ByteSub}(A_i + A_j)$,

    for $i, j = 0, \ldots, 15$
  - **bijective**, i.e., there exists a one-to-one mapping of input and output bytes

    $\Rightarrow$ S-Box can be uniquely reversed



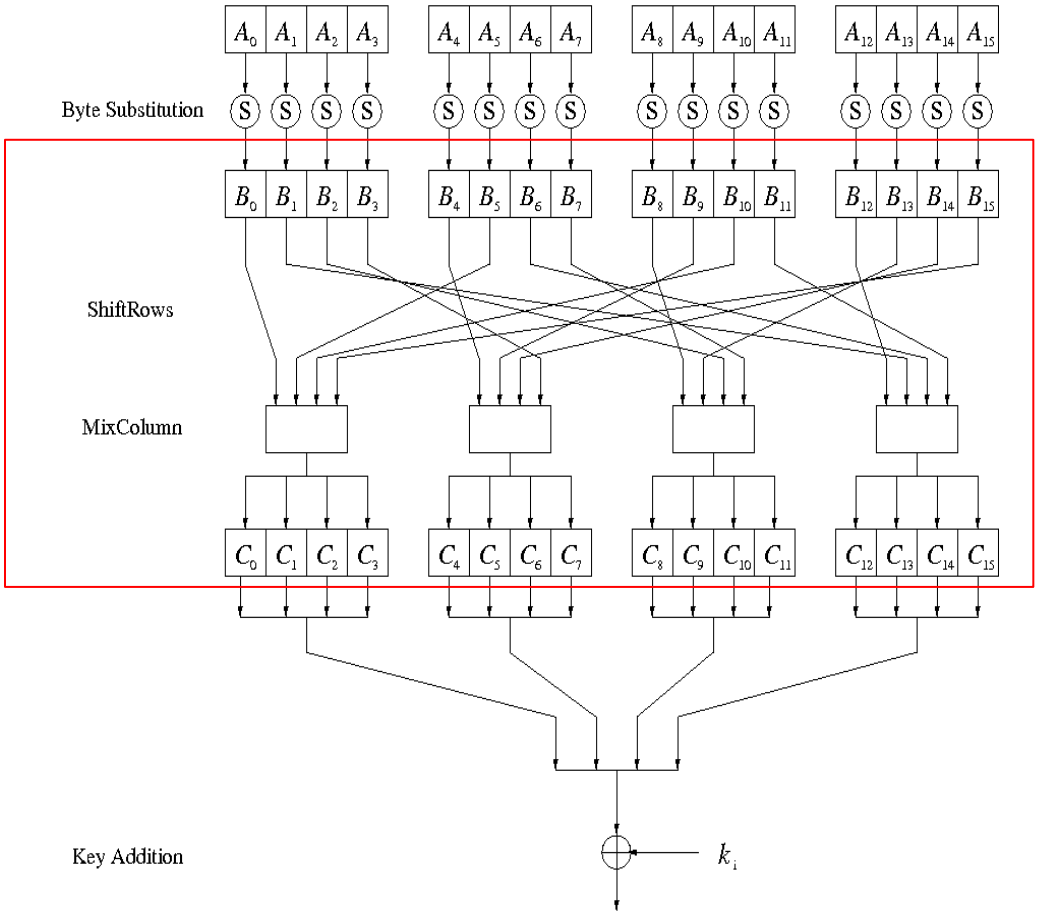
$$S(x) = Ax^{-1} + b$$

- In software implementations, the S-Box is usually realized as a lookup table

Diffusion Layer

The Diffusion layer

- provides diffusion over all input state bits
- consists of two sublayers:
  - **ShiftRows Sublayer**: Permutation of the data on a byte level
  - **MixColumn Sublayer**: Matrix operation which combines ("mixes") blocks of four bytes
- performs a linear operation on state matrices *A*, *B*, i.e.,



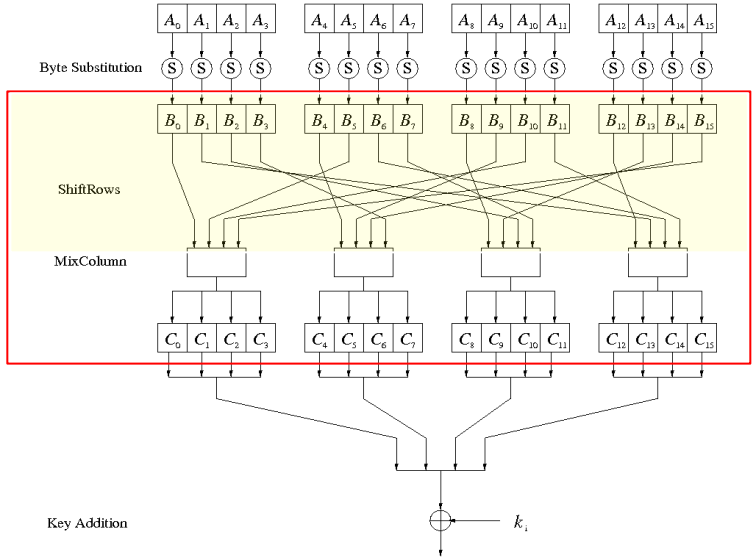
$$\mathrm{DIFF}(A) + \mathrm{DIFF}(B) = \mathrm{DIFF}(A + B)$$

ShiftRows Sublayer

- Rows of the state matrix are shifted cyclically:

Input matrix

B_0	B_4	B_8	B_{12}
B_1	B_5	B_9	B_{13}
B_2	B_6	B_{10}	B_{14}
B_3	B_7	B_{11}	B_{15}



Output matrix

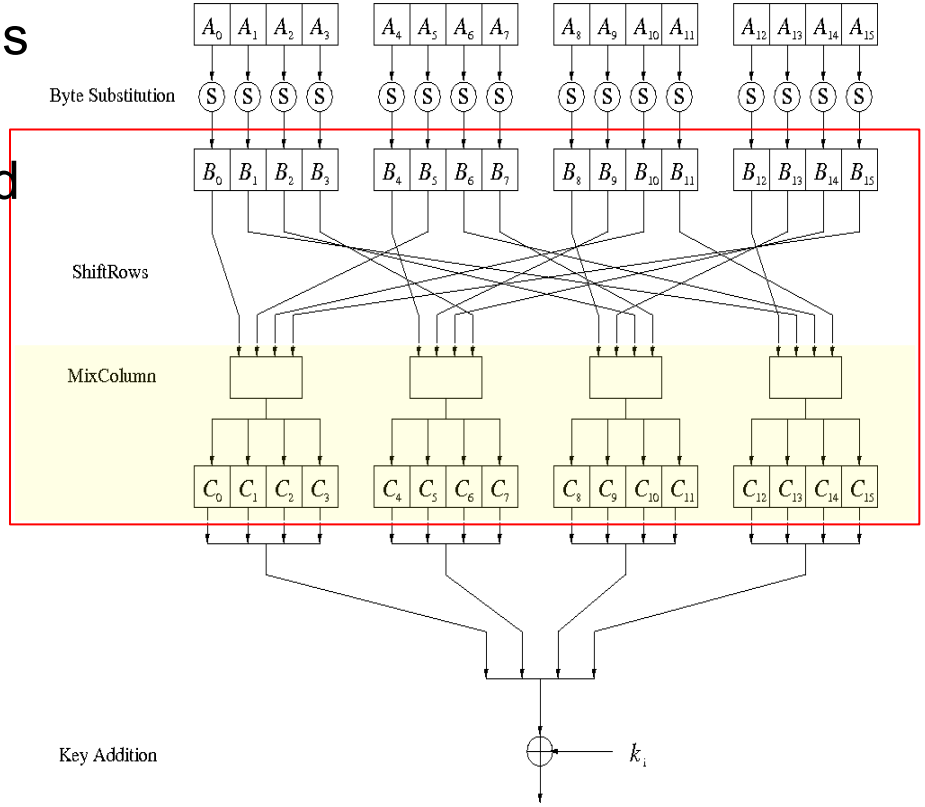
B_0	B_4	B_8	B_{12}	no shift
B_5	B_9	B_{13}	B_1	← one position left shift
B_{10}	B_{14}	B_2	B_6	← two positions left shift
B_{15}	B_3	B_7	B_{11}	← three positions left shift

MixColumn Sublayer

- Linear transformation which mixes each column of the state matrix
- Each 4-byte column is considered as a vector and multiplied by a fixed 4×4 matrix, e.g.,

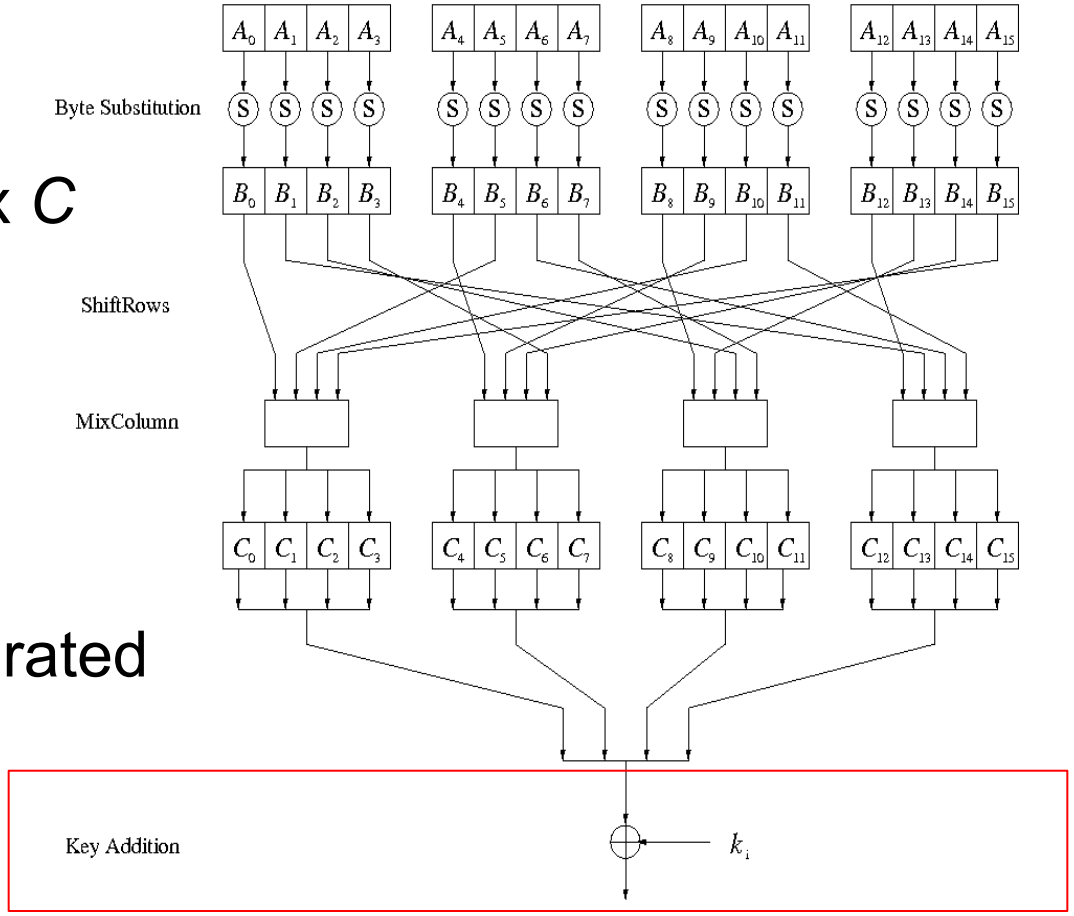
$$\begin{pmatrix} C_0 \\ C_1 \\ C_2 \\ C_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \cdot \begin{pmatrix} B_0 \\ B_5 \\ B_{10} \\ B_{15} \end{pmatrix}$$

where 01, 02 and 03 are given in hexadecimal notation



Key Addition Layer

- Inputs:
  - 16-byte state matrix C
  - 16-byte subkey k_i
- Output: $C \oplus k_i$
- The subkeys are generated in the key schedule



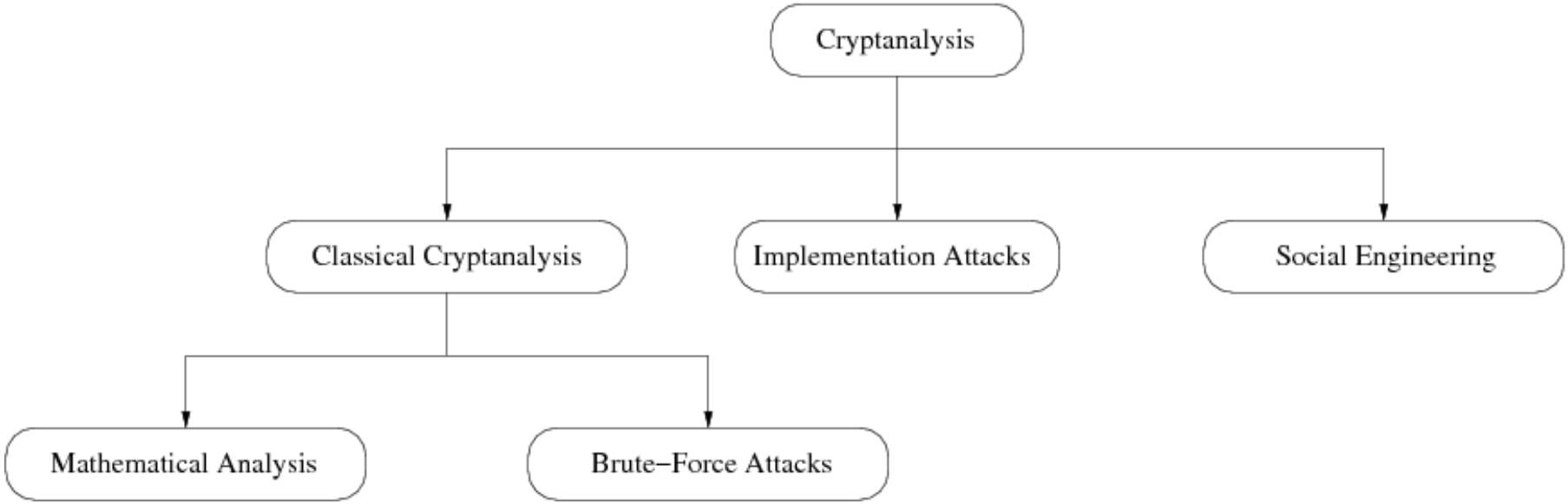
Implementation in Software

- One requirement of AES was the possibility of an efficient software implementation
- Straightforward implementation is well suited for 8-bit processors (e.g., smart cards), but inefficient on 32-bit or 64-bit processors
- A more sophisticated approach: Merge all round functions (except the key addition) into one table look-up
  - This results in four tables with 256 entries, where each entry is 32 bits wide
  - One round can be computed with 16 table look-ups
- Typical SW speeds are more than 1.6 Gbit/s on modern 64-bit processors

Security

- **Brute-force attack:** Due to the key length of 128, 192 or 256 bits, a brute-force attack is not possible
- **Analytical attacks:** There is no analytical attack known that is better than brute-force
- **Side-channel attacks:**
  - Several side-channel attacks have been published
  - Note that side-channel attacks do not attack the underlying algorithm but the implementation of it
- In 2003, the US National Security Agency (NSA) announced that it allows AES to encrypt classified documents up to the level SECRET for all key lengths, and up to the TOP SECRET level for key lengths of either 192 or 256 bits.

Cryptanalysis: Attacking Cryptosystems



- **Classical Attacks**
  - Mathematical Analysis
  - Brute-Force Attack
- **Implementation Attack**: Try to extract key through reverse engineering or power measurement, e.g., for a banking smart card
- **Social Engineering**: E.g., trick a user into giving up password

Why do we need Cryptanalysis?

- There is no *mathematical proof of security* for any practical cipher
- The only way to have assurance that a cipher is secure is to try to break it (and fail)

Kerckhoff's Principle is paramount in modern cryptography:

> A cryptosystem should be secure even if the attacker (Eve) knows all details about the system, with the exception of the secret key

- In order to achieve Kerckhoff's Principle in practice: **Only use widely known ciphers that have been cryptanalyzed for several years by good cryptographers!** (*Understanding Cryptography* only treats such ciphers)
- **Remark:** It is tempting to assume that a cipher is "more secure" if its details are kept secret. However, history has shown time and again that secret ciphers can almost always been broken once they have been reversed engineered. (Example: Crypto1 on the MIFARE Classic system.)

Brute-Force Attack (or Exhaustive Key Search) against Symmetric Ciphers

- Treats the cipher as a black box
- Requires (at least) 1 plaintext-ciphertext pair (x_0, y_0)
- Check all possible keys until condition is fulfilled:

$$d_K(y_0) \stackrel{?}{=} x_0$$

- How many keys do we need ?

Key length in bit	Key space	Security life time (assuming brute-force as best possible attack)
64	2^{64}	**Short term** (few days or less)
128	2^{128}	**Long-term** (several decades in the absence of quantum computers)
256	2^{256}	**Long-term** (also resistant against quantum computers – note that QC do not exist at the moment and might never exist)

Important: An adversary only needs to succeed with **one** attack. Thus, a long key space does not help if other attacks (e.g., social engineering) are possible.

Key Lengths and Security Levels

Symmetric	*ECC*	*RSA, DL*	*Remark*
64 Bit	128 Bit	≈ 700 Bit	Only short term security (a few hours or days)
80 Bit	160 Bit	≈ 1024 Bit	Medium security (except attacks from big governmental institutions etc.)
128 Bit	256 Bit	≈ 3072 Bit	Long term security (without quantum computers)

- The exact complexity of RSA (factoring) and DL (Index-Calculus) is difficult to estimate
- The existence of quantum computers would probably be the end for ECC, RSA & DL (at least 2-3 decades away, and some people doubt that QC will ever exist)